

Data Analysis

We analysis the data to understand the data and how we can see the pattern and trend about the data. To process the data we examine the data, cleaning, transforming and extract the data in order to get the meaningful or useful information.

```
#Load and preview the data
users_data = pd.read_pickle('users_data_cleaned.pickle')
users_data.head(n = 5)
```

Python

	user_id	number_transactions	total_amount_usd	age	job	marital	education	default	housing	loan	contact	duration	campaign	pdays
0	9231c446-cb16-4b2b-a7f7-ddfc8b25aaf6	3.0	2143.0	58	management	married	tertiary	False	True	False	None	261	1	-1
1	bb92765a-08de-4963-b432-496524b39157	NaN	NaN	44	technician	single	secondary	False	True	False	None	151	1	-1
2	573de577-49ef-42b9-83da-d3cfb817b5c1	2.0	2.0	33	entrepreneur	married	secondary	False	True	True	None	76	1	-1
3	d6b66b9d-7c8f-4257-a682-e136f640b7e3	NaN	NaN	47	blue-collar	married	None	False	True	False	None	92	1	-1
4	fade0b20-7594-4d9a-84cd-c02f79b1b526	1.0	1.0	33	None	single	None	False	False	False	None	198	1	-1

- Load data from pickle file 'user_data_cleaned'
- Display data with 5 row only

```
#Check the shape of the data
users_data.shape
```

Python

```
[2]
... (45209, 19)
```

```
#Check the number of unique users
len(np.unique(users_data.user_id))
```

Python

```
[3]
... 45209
```

- Check the total row and column of the data
- Row= 45209 and column 19
- Check unique data for user id with the result = 45209

```
#Check the data types
users_data.info()
users_data.columns.to_series().groupby(users_data.dtypes).groups
```

Python

```
<class 'pandas.core.frame.DataFrame'>
Index: 45209 entries, 0 to 45215
Data columns (total 19 columns):
#   Column                Non-Null Count  Dtype
---  -
0   user_id                45209 non-null  object
1   number_transactions    35210 non-null  float64
2   total_amount_usd       35210 non-null  float64
3   age                   45209 non-null  int64
4   job                   44921 non-null  object
5   marital               45209 non-null  object
6   education             43352 non-null  object
7   default               45209 non-null  bool
8   housing               45209 non-null  bool
9   loan                  45209 non-null  bool
10  contact               32191 non-null  object
11  duration              45209 non-null  int64
12  campaign              45209 non-null  int64
13  pdays                45209 non-null  int64
14  previous              45209 non-null  int64
15  poutcome              8252 non-null   object
16  term_deposit          45209 non-null  bool
17  date_joined           45179 non-null  datetime64[ns]
18  device                45115 non-null  object
dtypes: bool(4), datetime64[ns](1), float64(2), int64(5), object(7)
memory usage: 5.7+ MB
```

```
{bool: ['default', 'housing', 'loan', 'term_deposit'], int64: ['age', 'duration', 'campaign', 'pdays', 'previous'], float64: ['number_transactions', 'total_amount_usd']}
```

- Check the data info column name and data type for each column
- Total data type of data

```
#Check for correlations
users_data.corr(numeric_only = True).abs()
```

Python

	number_transactions	total_amount_usd	age	default	housing	loan	duration	campaign	pdays	previous	term_deposit
number_transactions	1.000000	0.163409	0.008813	0.138838	0.030429	0.075319	0.017220	0.026431	0.030751	0.023046	0.053390
total_amount_usd	0.163409	1.000000	0.095839	0.065390	0.066857	0.084526	0.022586	0.017274	0.006435	0.016952	0.050785
age	0.008813	0.095839	1.000000	0.017875	0.185552	0.015641	0.004645	0.004767	0.023745	0.001297	0.025168
default	0.138838	0.065390	0.017875	1.000000	0.006020	0.077232	0.010017	0.016819	0.029982	0.018331	0.022421
housing	0.030429	0.066857	0.185552	0.006020	1.000000	0.041341	0.005041	0.023583	0.124197	0.037087	0.139161
loan	0.075319	0.084526	0.015641	0.077232	0.041341	1.000000	0.012395	0.009972	0.022762	0.011048	0.068193
duration	0.017220	0.022586	0.004645	0.010017	0.005041	0.012395	1.000000	0.084551	0.001549	0.001213	0.394549
campaign	0.026431	0.017274	0.004767	0.016819	0.023583	0.009972	0.084551	1.000000	0.088636	0.032860	0.073179
pdays	0.030751	0.006435	0.023745	0.029982	0.124197	0.022762	0.001549	0.088636	1.000000	0.454817	0.103616
previous	0.023046	0.016952	0.001297	0.018331	0.037087	0.011048	0.001213	0.032860	0.454817	1.000000	0.093232
term_deposit	0.053390	0.050785	0.025168	0.022421	0.139161	0.068193	0.394549	0.073179	0.103616	0.093232	1.000000

- Check correlation between data
- Check for numeric only
- +1 = positive relationship, -1 = negative relationship , 0 = no relationship
- Example **positive relationship** : **pdays** and **previous** : **0.45817**. it is the highest and nearest to 1 . means if the increasing the variable then the others variable would be increasing also.
- Example **no relationship** or **weak relationship** : **number_transactions** and **age** : **0.008813**. it is the lowest and nearest to 0 . means if 0 no relationship between 2 variable or no effect if there have changes between 2 variables.
- Example for **simple relationship** : **housing** and **age** : **0.185552**, it is moderate relationship. The older age can be have their own house.

```
#Generate summary statistics for all of the data
users_data.describe(include = 'all')
```

Python

	user_id	number_transactions	total_amount_usd	age	job	marital	education	default	housing	loan	contact	duration	can
count	45209	35210.000000	35210.000000	45209.000000	44921	45209	43352	45209	45209	45209	32191	45209.000000	45209.0
unique	45209	NaN	NaN	NaN	11	3	3	2	2	2	2	NaN	
top	5905741c-3f30-416f-8ee5-71abad035de6	NaN	NaN	NaN	blue-collar	married	secondary	False	True	False	cellular	NaN	
freq	1	NaN	NaN	NaN	9731	27212	23202	44394	25128	37965	29285	NaN	
mean	NaN	3.977052	1369.417751	40.935853	NaN	NaN	NaN	NaN	NaN	NaN	NaN	258.153067	2.0
min	NaN	1.000000	-8019.000000	18.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0.000000	1.0
25%	NaN	2.000000	73.000000	33.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	103.000000	1.0
50%	NaN	3.000000	451.000000	39.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	180.000000	2.0
75%	NaN	4.000000	1438.000000	48.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	319.000000	3.0
max	NaN	20.000000	102127.000000	95.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	4918.000000	63.0
std	NaN	3.814329	3063.412688	10.618653	NaN	NaN	NaN	NaN	NaN	NaN	NaN	257.525446	3.0

- Check info data about count,unique,top,frequent,mean,min,25%,50%,75%, max and standard for all features
- The purpose to gain the overview of dataset and understand the data before we go to further analysis
- We can also identify the missing value, outliers and error.
- To understand how the data spread and its central tendencies
- Help to transform data and feature selection
- Detect anomalies in min, max and unexpected top values
- **Count** for column **user_id** have 45209 and for **unique** also 45209. From the data there no **duplicate** data.
- **Top** value is the most frequent occurring in column like **job** which is 'blue-collar' and the **freq** is the total data of the most common in **job** is about 9731
- **Mean** is the average value which is central tendency. For example, the **number_transactions** mean is 3.977052 and the **standard deviation** is 3.814329 which is the measurement of spread data.
- **Min** for **number_transactions** is 1 and **Max** is 20
- **25%** for **number_transactions** is 2 which is (**First Quartile, Q1**) – The value below which 25% of the data falls
- **50%** for **number_transactions** is 3 which is (**Median, Q2**) – The middle value of the dataset.
- **75%** for **number_transactions** is 4 which is (**Third Quartile, Q3**) – The value below which 75% of the data falls.
- The dataset contains both **numerical** (age, number_transactions, total_amount_usd, duration) and **categorical** (job, marital, education, default, housing, loan, contact) features.
- Some columns have missing values (NaN), such as job, education, and contact. This is because this feature is not a numerical.
- There is a **large variation in financial transactions**, suggesting different user spending patterns. For example, **total_amount_usd** the mean transaction is 1369.42 USD and standard deviation is **3063.41 USD**. Which it means that the many transactions are far from average transaction and some users may have spent very little, while others have spent large amounts.

- **Potential data issues:** Negative values in **total_amount_usd** may need further investigation. For example, min is -8019.00 USD (possibly due to refunds or errors)
- This statistical summary is **useful for exploratory data analysis (EDA)** before applying further transformations or machine learning models.

```
#Generate summary statistics for numerical data only
users_data.describe(include = np.number)
```

	number_transactions	total_amount_usd	age	duration	campaign	pdays	previous
count	35210.000000	35210.000000	45209.000000	45209.000000	45209.000000	45209.000000	45209.000000
mean	3.977052	1369.417751	40.935853	258.153067	2.763897	40.199651	0.580349
std	3.814329	3063.412688	10.618653	257.525446	3.098076	100.130586	2.303489
min	1.000000	-8019.000000	18.000000	0.000000	1.000000	-1.000000	0.000000
25%	2.000000	73.000000	33.000000	103.000000	1.000000	-1.000000	0.000000
50%	3.000000	451.000000	39.000000	180.000000	2.000000	-1.000000	0.000000
75%	4.000000	1438.000000	48.000000	319.000000	3.000000	-1.000000	0.000000
max	20.000000	102127.000000	95.000000	4918.000000	63.000000	871.000000	275.000000

- Check info data about count, unique, top, frequent, mean, 25%, 50%, 75%, max and standard for numeric features only
- From the data, we can compare between **number_transaction** and **total_amount_usd** for count is 35210 is less than the rest column which is **age, duration, campaign, pdays** and **previous** that were 45209. So, it means that the column for **number_transaction** and **total_amount** got the missing data.

```
#Generate modal values for all data
# Drop user ID since it's unique.
users_data.drop(['user_id'], axis = 1).mode()
```

	number_transactions	total_amount_usd	age	job	marital	education	default	housing	loan	contact	duration	campaign	pdays	previous	poutcome
0	2.0	0.0	32.0	blue-collar	married	secondary	False	True	False	cellular	124.0	1.0	-1.0	0.0	failure
1	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

- Check mode data. Before that remove the user_id since it is unique value it to see the mode of data
- Total mode of data is 2

```

> ~
• #Generate skewness and kurtosis measurements
  users_data.skew(numeric_only = True)

[10] ✓ 0.0s Python
...
number_transactions    2.704543
total_amount_usd      8.596128
age                   0.684861
default               7.245206
housing              -0.224686
loan                  1.852545
duration              3.144556
campaign              4.898555
pdays                2.615635
previous              41.845672
term_deposit          2.383403
dtype: float64

```

- **Skewness** measures how **asymmetric** the distribution is
- **Skewness = 0** → Perfectly symmetric (normal distribution).
- **Positive Skew (> 0)** → Right-skewed (long tail on the right).
- **Negative Skew (< 0)** → Left-skewed (long tail on the left).
- **Example** positive Skewness, **total_amount_usd** and **default** is the most right skewness and following **campaign**, **duration**, **number_transactions**, **pdays** and **term_deposit**. These are consider moderate right skewness. The higher on the right skewness is **previous** but it can be has a **outliers**.
- **Example** Negative Skewness, **housing** -0.224686 is negative skewness.
- **Example** almost to normal skewness, **age** 0.684861.

```

> ~
  users_data.kurt(numeric_only = True)

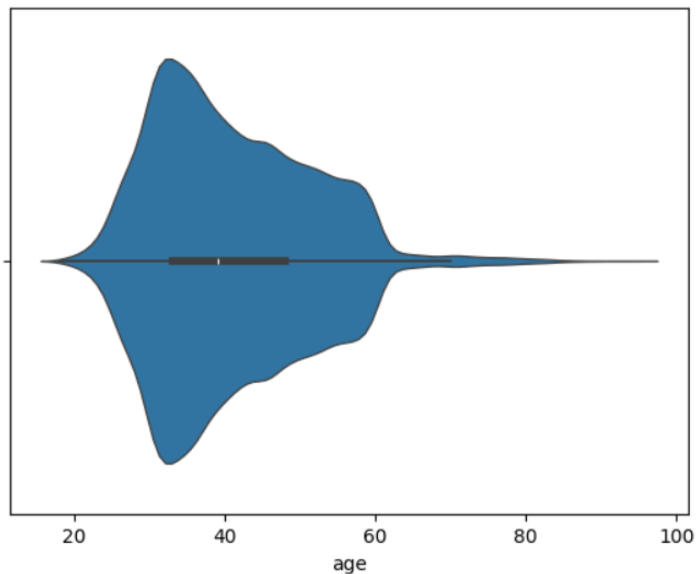
11] ✓ 0.0s Python
...
number_transactions    6.659034
total_amount_usd      150.790967
age                   0.319760
default               50.495241
housing              -1.949602
loan                  1.431987
duration              18.155941
campaign              39.248145
pdays                6.934713
previous              4506.684640
term_deposit          3.680770
dtype: float64

```

- **Kurtosis** measures whether the data has heavy tails (outliers) or is flat (evenly spread out).
- **Kurtosis = 3** → Normal distribution (**mesokurtic**).
- **Kurtosis > 3** → Heavy tails, more extreme values (**leptokurtic**).
- **Kurtosis < 3** → Light tails, fewer extreme values (**platykurtic**).
- **total_amount_usd** (Very high) → Some users have very large transaction amounts compared to others.
- **previous** (4506.68) → Extremely high kurtosis, indicating most users had 0 previous contacts, but a few had many.
- **campaign** (39.24) → Suggests most users were contacted few times, but some were contacted a lot.
- **duration** (18.15) → Suggests most contacts were short, but some were extremely long.
- Transaction-related variables (**total_amount_usd**, **number_transactions**) have **strong right-skew and high kurtosis**, meaning a small group of users make very high-value transactions.
- **Duration** and **campaign** are also right-skewed, meaning most contacts were short, but some were very long.
- Extreme **outliers** exist in **previous** and **total_amount_usd**


```
#Generate a violin plot for age
sns.violinplot(x = users_data['age'], linewidth = 0.9);
```

✓ 0.3s



- Violin plot for age and it is combining between the boxplot and density plot to gain more information about the spreading of data
- The white dot is the median or Q2
- The black box/bar in between white dot is IQR which is span between Q1 and Q3
- Thin bar after or before black box/bar indicate for min and max. for min is 20 and max is 90
- The wider section is representing the most data individual age in that range which is 30 to 50 years old
- The narrower section is the least or fewer people in the range from 60 to 90
- The outlier is more than 70 and so on
- The right tail is longer and it showing this is positive skewness and yes, it is minority.
- The comparison between box plot is it can show the outliers and for violin plot it can show a few the number of people age in the range. It also can help to easily to understand of data density.

```

#Generate a box plot for number_transactions
users_data['number_transactions'].describe()

```

✓ 0.0s

```

count    35210.000000
mean      3.977052
std       3.814329
min       1.000000
25%       2.000000
50%       3.000000
75%       4.000000
max       20.000000
Name: number_transactions, dtype: float64

```

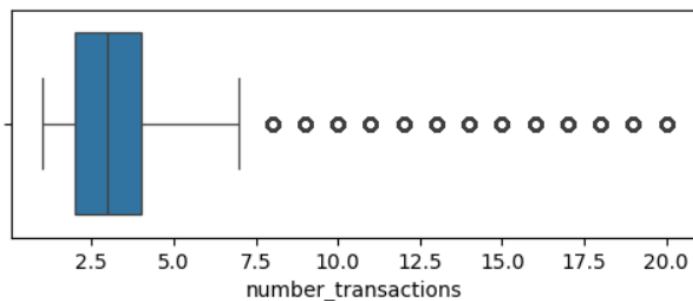
- Check info data about count, unique, top, frequent, mean, 25%, 50%, 75%, max and standard for **number_transactions** features only

```

plt.figure(figsize = (6, 2))
sns.boxplot(x = users_data['number_transactions'],
            linewidth = 0.9);

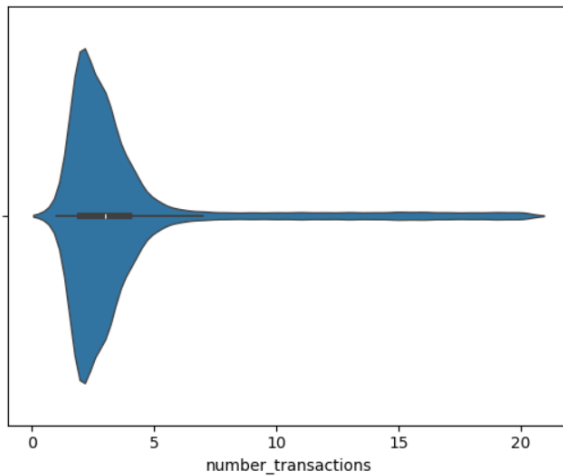
```

✓ 0.1s



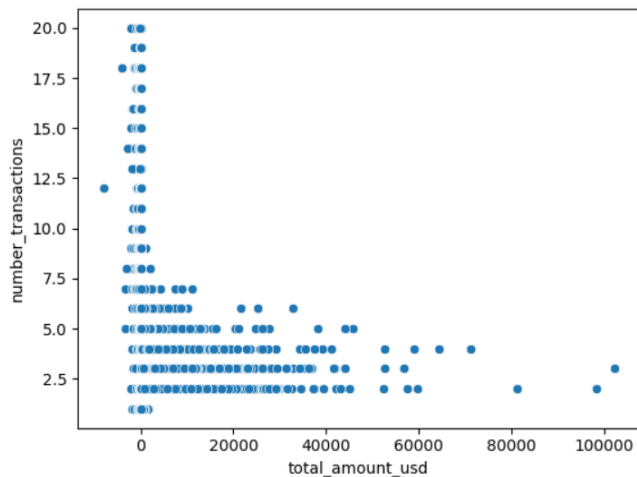
- Box plot for **number_transactions**
- Q1 = 2, Q2 = 3, Q3 = 4
- IQR = 2 to 4
- Min = 1 and max = 20
- Outliers start more than 7
- The Skewness is positive since the outliers are on the right

```
#Generate a violin plot for number_transactions
sns.violinplot(x = users_data['number_transactions'],
               linewidth=0.9);
```



- Thin bar after or before black box/bar indicate for min an max. for min is 1 and max is 20
- The wider section is representing the most number transaction in that range which is 2 to 4 transactions
- The narrower section is the least or fewer people in the range from 7 to 20
- The outlier is more than 7 and so on
- The right tail is longer and it showing this is positive skewness and yes, it is minority.

```
#Generate scatter plots comparing total_amount_usd to number_transactions
sns.scatterplot(data = users_data, x = 'total_amount_usd',
                y = 'number_transactions');
```



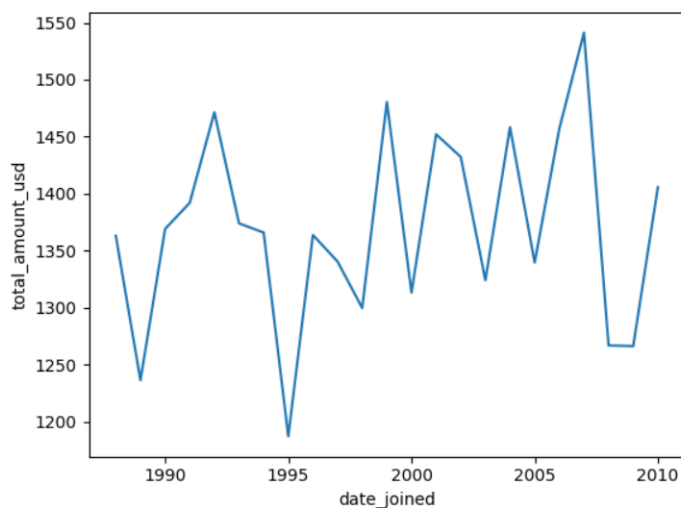
- This is scatter plot and it showing the relationship between **number_transactions** and **total_amount_usd**

- The data showing **majority** or the most data is around 1 to 5 transaction and 20000 amount USD
- The **minority** data is the least number transaction and the highest amount USD spend
- This shape of data or trend showing that this data is a negative between **number_transactions** and **total_amount_usd**. the highest number transaction the least amount USD to spend.
- These data explained that only least number of transactions will do higher amount transaction USD, meanwhile the large number of transactions will spending small amount transaction USD
- There no pattern showing a straight line, so it's not a linear
- So may have other column/field will influence/affected the data

```
#Generate a line plot for total_amount_usd
years = users_data['date_joined'].dt.year

sns.lineplot(data = users_data, x = years,
              y = 'total_amount_usd',
              estimator = np.mean,
              errorbar = None);
```

✓ 0.2s



- Used seaborn to make a line plot
- Showing relation between y=total_amount_usd and x=date_joined
- Line plot based on mean of total_amount_usd for a year

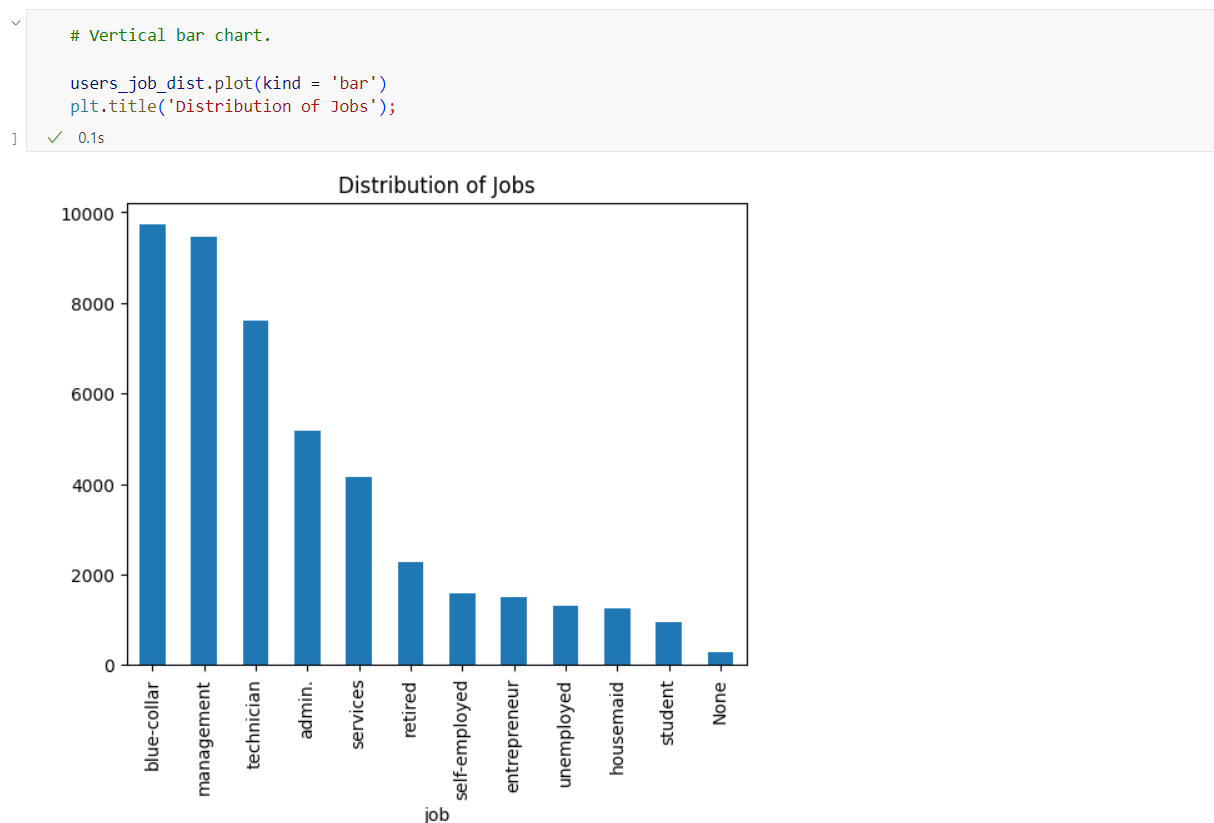
```
#Generate bar charts for job
users_job_dist = \
users_data['job'].value_counts(dropna = False)

users_job_dist
```

[18] ✓ 0.0s 🔗 Open 'users_job_dist' in Data Wrangler

```
...
job
blue-collar      9731
management      9457
technician       7597
admin.           5171
services         4154
retired          2264
self-employed    1579
entrepreneur     1487
unemployed       1303
housemaid        1240
student          938
None             288
Name: count, dtype: int64
```

- Check category value in this column for job and count for each category included None



- Show graph bar for total of each category job

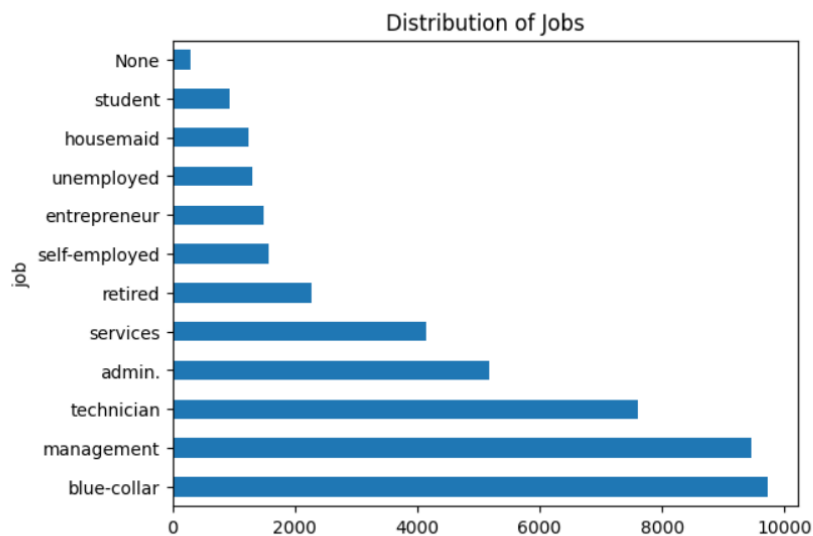
```

# Horizontal bar chart

users_job_dist.plot(kind = 'barh')
plt.title('Distribution of Jobs');

```

✓ 0.2s



- Show graph bar for total of each category job in horizontal way.
- Easy to read If long category name

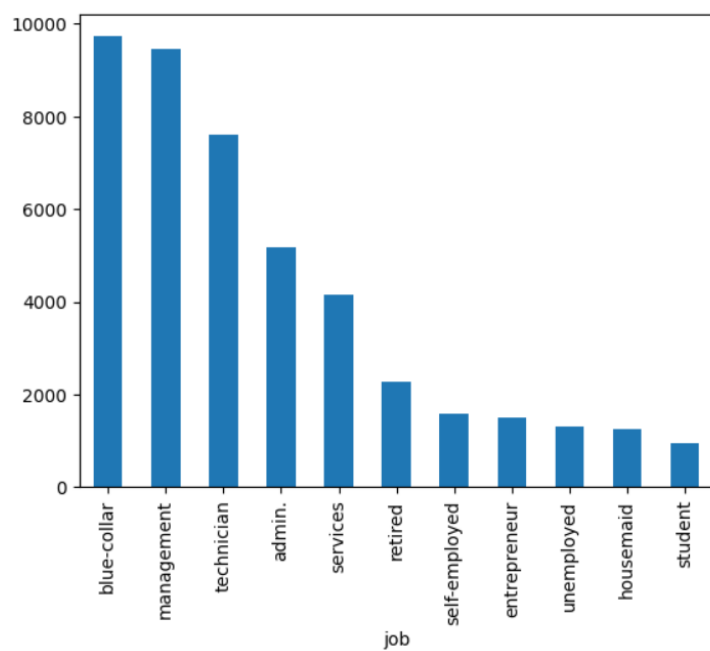
```

# Exclude missing values.

users_data['job'].value_counts().plot(kind = 'bar');

```

✓ 0.1s



- Check category value in this column for job and count for each category excluded None

```
#Generate a bar chart for marital
users_marital_dist = \
users_data['marital'].value_counts(dropna = False)

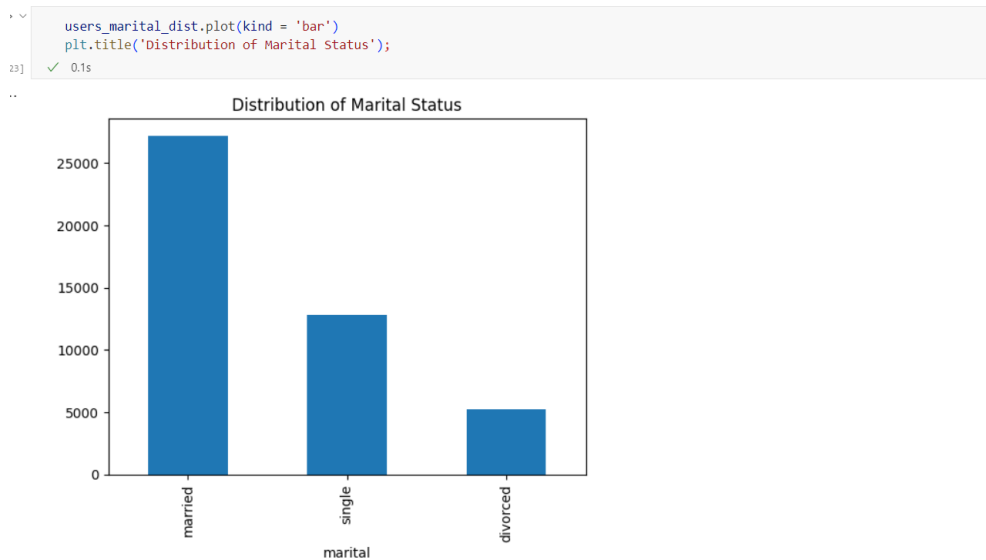
users_marital_dist
```

✓ 0.0s Open 'users_marital_dist' in Data Wrangler Pyth

marital	count
married	27212
single	12790
divorced	5207

Name: count, dtype: int64

- Check category value in this column for marital and count for each category included None/NAN/Null
- But this data don't have None/NAN/Null value



- Show graph bar for total of each category Marital Status

Generate Code Markdown

```
#Generate a bar chart for education
users_education_dist = \
users_data['education'].value_counts(dropna = False)

users_education_dist
```

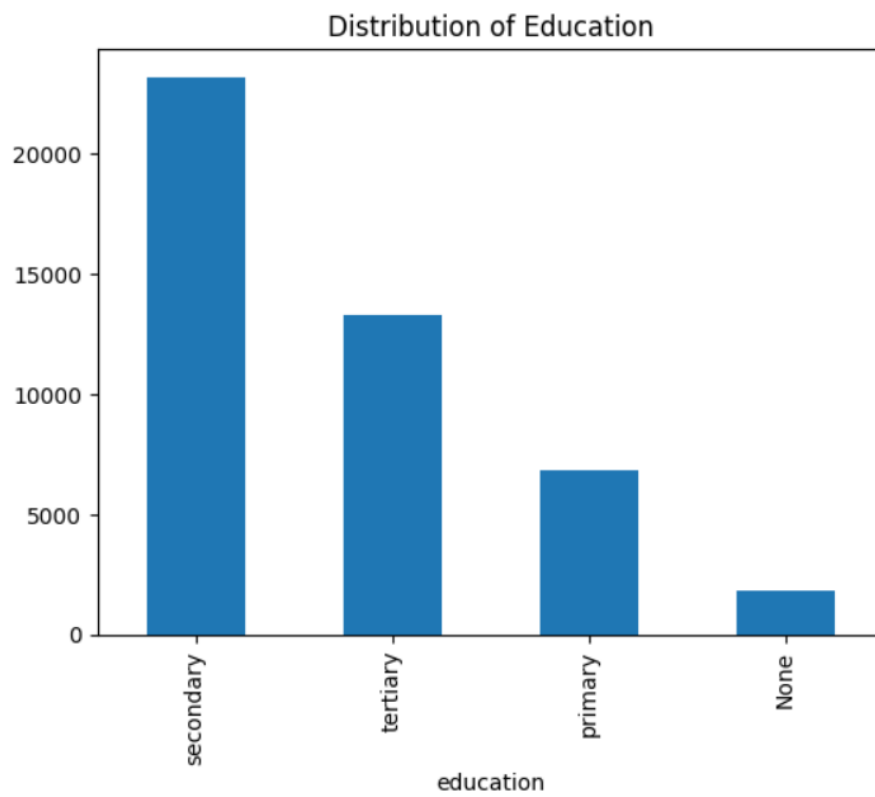
✓ 0.0s Open 'users_education_dist' in Data Wrangler

education	count
secondary	23202
tertiary	13300
primary	6850
None	1857

Name: count, dtype: int64

- Check category value in this column for **education** and count for each category included None

```
users_education_dist.plot(kind = 'bar')  
plt.title('Distribution of Education');  
] ✓ 0.1s
```

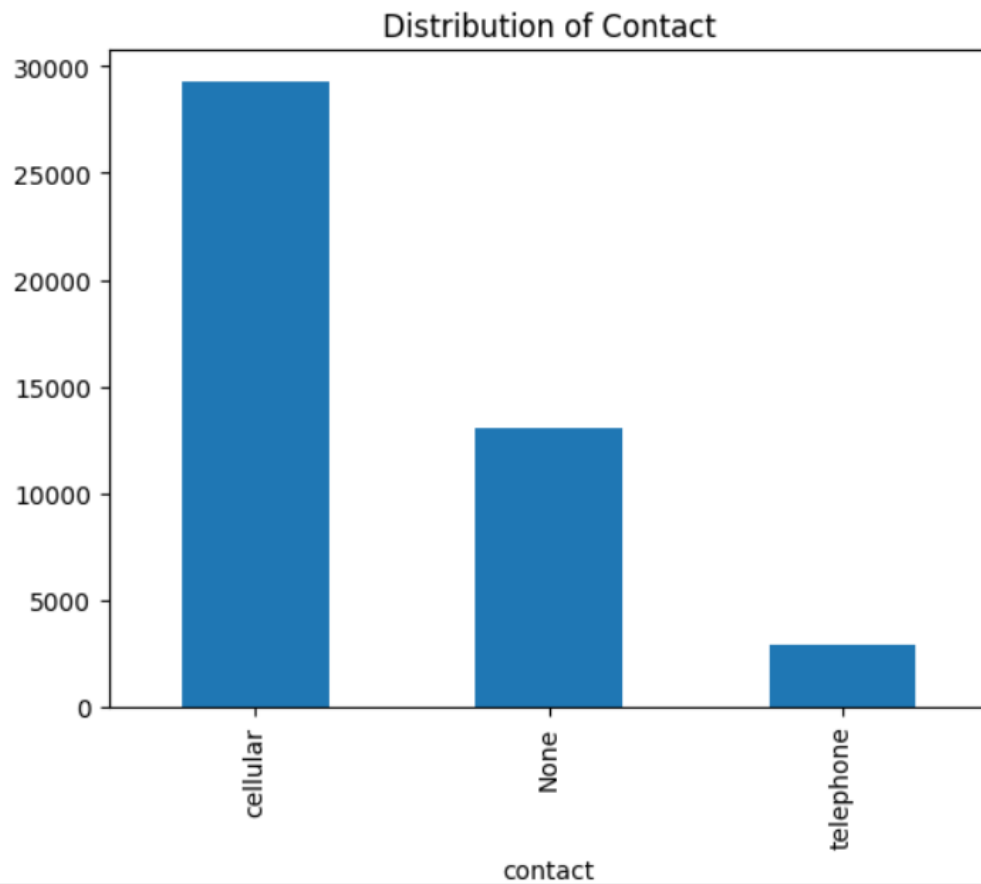


- Show graph bar for total of each category **Education**

```
#Generate a bar chart for contact
users_contact_dist = \
users_data['contact'].value_counts(dropna = False)

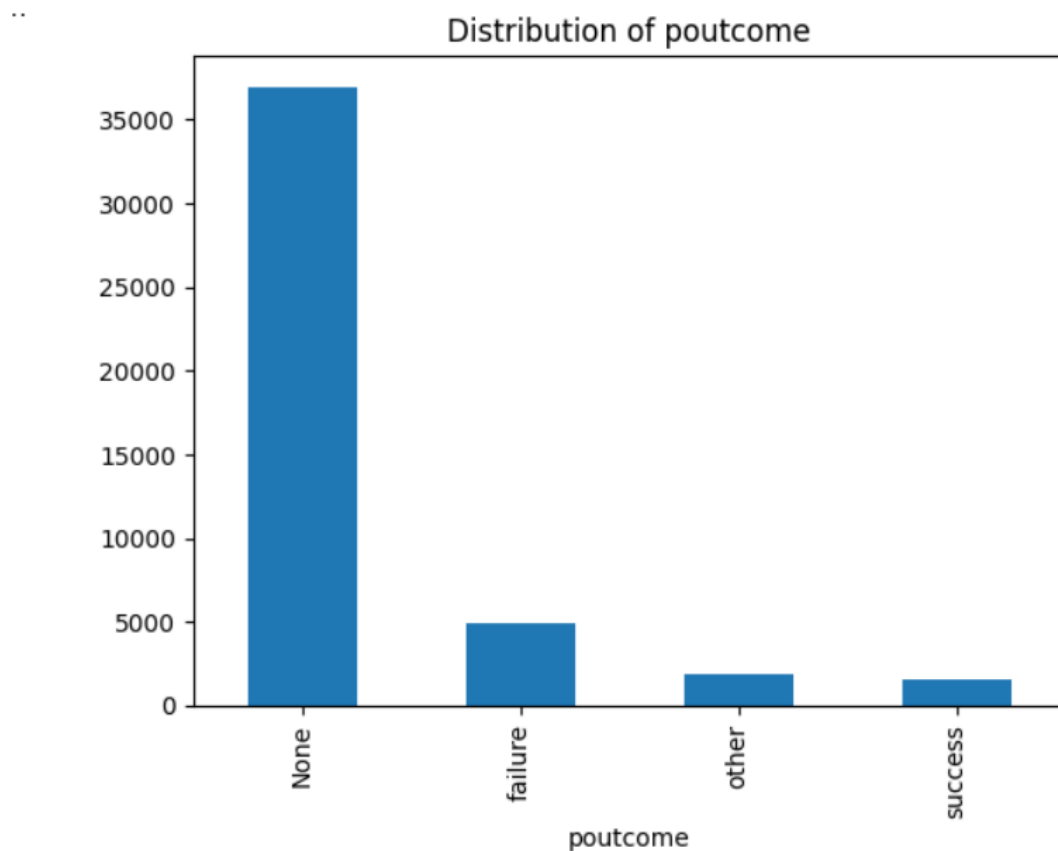
users_contact_dist.plot(kind = 'bar')
plt.title('Distribution of Contact');
```

✓ 0.1s



- Show graph bar for total of each category **contact**

```
> \n\n#Generate a bar chart for poutcome\nusers_poutcome_dist = \nusers_data['poutcome'].value_counts(dropna = False)\n\nusers_poutcome_dist.plot(kind = 'bar')\nplt.title('Distribution of poutcome');\n27] ✓ 0.2s
```

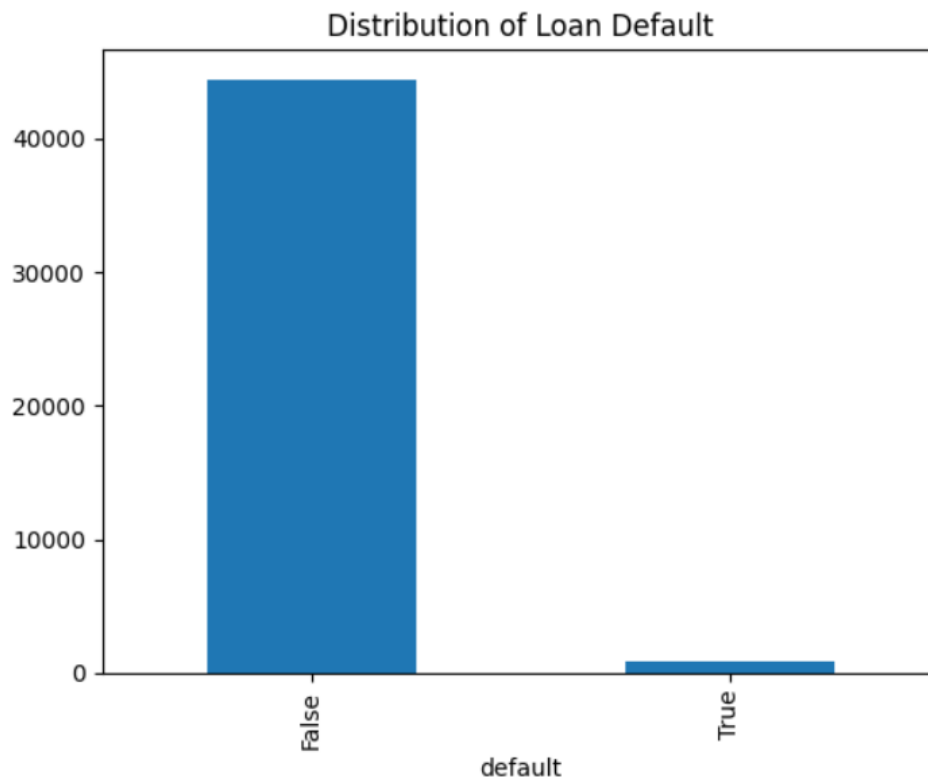


- Show graph bar for total of each category **poutcome**

```
#Generate a bar chart for default
users_default_dist = \
users_data['default'].value_counts(dropna = False)

users_default_dist.plot(kind = 'bar')
plt.title('Distribution of Loan Default');
```

✓ 0.1s



- Show graph bar for total of each category **default**

```
#Generate a bar chart for device
users_device_dist = \
users_data['device'].value_counts(dropna = False)

users_device_dist
```

✓ 0.0s [Open 'users_device_dist' in Data Wrangler](#)

```
device
mobile    27504
desktop   12112
tablet     5499
None         94
Name: count, dtype: int64
```

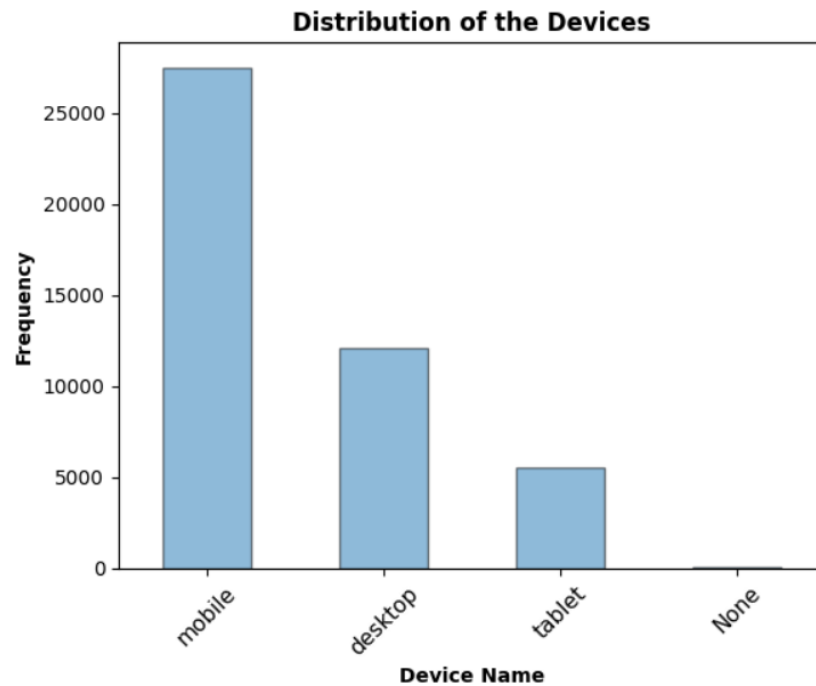
- Check category value in this column for **device** and count for each category included None

```

users_device_dist.plot(kind = 'bar',
                        alpha = 0.5, edgecolor = 'black')
plt.title('Distribution of the Devices',
          size = 12, weight = 'bold')
plt.xticks(rotation = 45, size = 11)
plt.xlabel('Device Name', size = 10, weight = 'bold')
plt.ylabel('Frequency', size = 10, weight = 'bold')
plt.show();

```

✓ 0.1s



- Show graph bar for total of each category **device** included None
- The graph bar adds more attributes
- Title – ‘Distribution of the device’, Bold, font size = 12
- X label – name= ‘Device Name’, Bold, font size = 10
- X ticks – rotation = 45, font size = 11
- Y label – name ‘Frequency’, Bold, font size = 10

```

#Generate a heatmap for the feature correlations
corr_matrix = users_data.corr(numeric_only = True)

```

corr_matrix

✓ 0.0s Open 'corr_matrix' in Data Wrangler

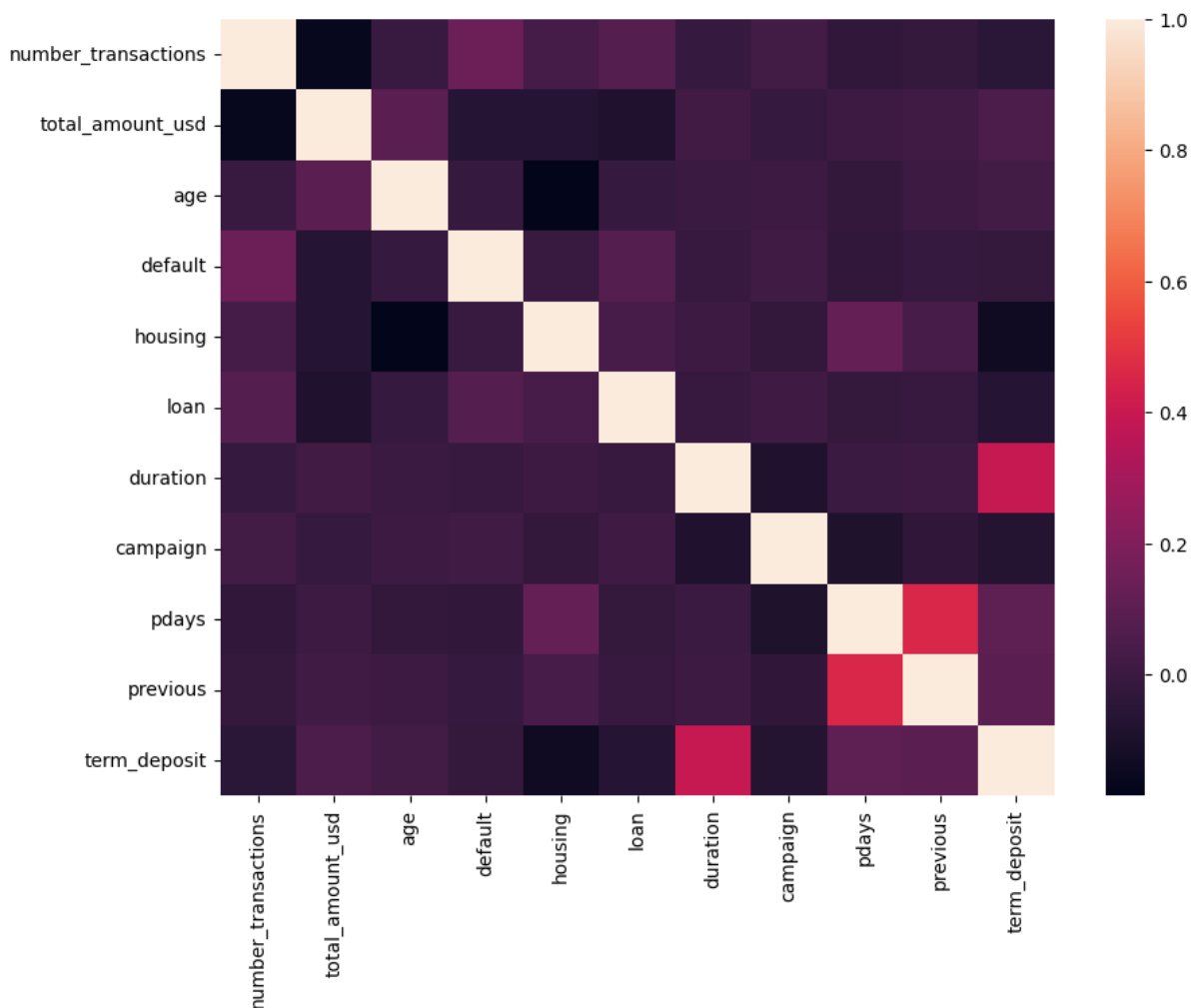
Python

	number_transactions	total_amount_usd	age	default	housing	loan	duration	campaign	pdays	previous	term_deposit
number_transactions	1.000000	-0.163409	-0.008813	0.138838	0.030429	0.075319	-0.017220	0.026431	-0.030751	-0.023046	-0.053390
total_amount_usd	-0.163409	1.000000	0.095839	-0.065390	-0.066857	-0.084526	0.022586	-0.017274	0.006435	0.016952	0.050785
age	-0.008813	0.095839	1.000000	-0.017875	-0.185552	-0.015641	-0.004645	0.004767	-0.023745	0.001297	0.025168
default	0.138838	-0.065390	-0.017875	1.000000	-0.006020	0.077232	-0.010017	0.016819	-0.029982	-0.018331	-0.022421
housing	0.030429	-0.066857	-0.185552	-0.006020	1.000000	0.041341	0.005041	-0.023583	0.124197	0.037087	-0.139161
loan	0.075319	-0.084526	-0.015641	0.077232	0.041341	1.000000	-0.012395	0.009972	-0.022762	-0.011048	-0.068193
duration	-0.017220	0.022586	-0.004645	-0.010017	0.005041	-0.012395	1.000000	-0.084551	-0.001549	0.001213	0.394549
campaign	0.026431	-0.017274	0.004767	0.016819	-0.023583	0.009972	-0.084551	1.000000	-0.088636	-0.032860	-0.073179
pdays	-0.030751	0.006435	-0.023745	-0.029982	0.124197	-0.022762	-0.001549	-0.088636	1.000000	0.454817	0.103616
previous	-0.023046	0.016952	0.001297	-0.018331	0.037087	-0.011048	0.001213	-0.032860	0.454817	1.000000	0.093232
term_deposit	-0.053390	0.050785	0.025168	-0.022421	-0.139161	-0.068193	0.394549	-0.073179	0.103616	0.093232	1.000000

- Find correlation all features for numeric only

```
fig = plt.figure(figsize = (10, 7.5))
sns.heatmap(corr_matrix);
```

✓ 0.3s



- Use seaborn library to show heat map graph to easier to read
- Used figsize 10 inc and 7.5 inch to make it bigger. The size also can make it clear to read

```
#Format the heatmap to make it easier to read
fig = plt.figure(figsize = (11, 8))

sns.heatmap(corr_matrix,
            cmap = 'seismic',
            linewidth = 0.75,
            linecolor = 'black',
            cbar = True,
            vmin = -1,
            vmax = 1,
            annot = True,
            annot_kws = {'size': 8, 'color': 'black'})

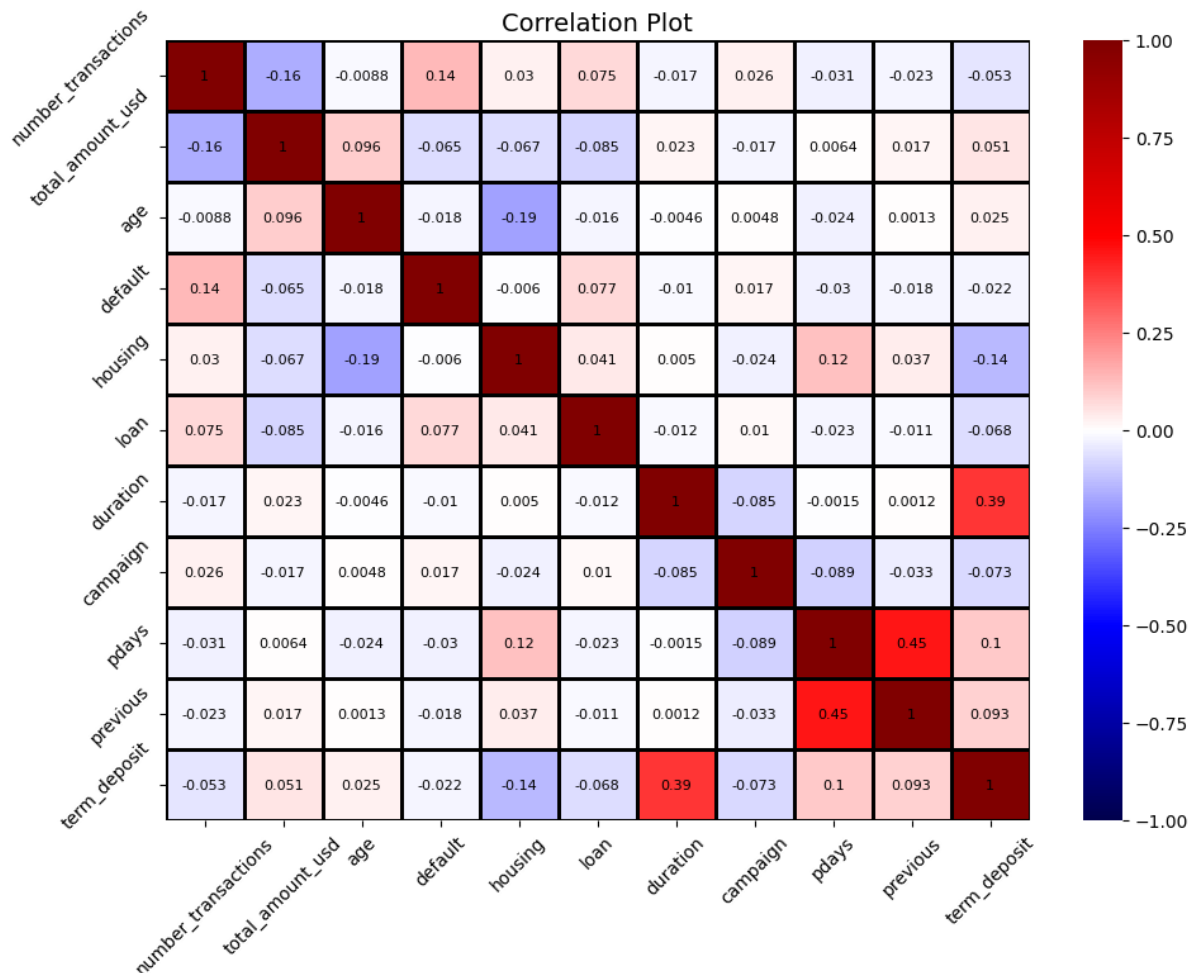
plt.tick_params(labelsize = 10, rotation = 45)
plt.title('Correlation Plot', size = 14);
```

✓ 0.5s

Pytho

- Add more attribute heatmap
- Cmap – seismic -> red to blue
- Red positive , blue – negative, white – no relationship
- Linewidth set to 0.75, the thick of line is 0.75

- Line color to black
- Vmin = -1 and vmax = 1, means the number will be in the range to -1and 1 only
- Annot = true , show number inside box
- Annot_kws size 8 and black for font size and color



```
#Identify missing values
users_data.isnull().sum()
✓ 0.0s
```

Python

```
user_id          0
number_transactions  9999
total_amount_usd  9999
age              0
job             288
marital         0
education       1857
default         0
housing         0
loan            0
contact        13018
duration        0
campaign        0
pdays         0
previous        0
poutcome       36957
term_deposit    0
date_joined     30
device          94
dtype: int64
```

- Check data null/nan if true will count the total data

```
#Identify the percentage of missing values for each feature
percent_missing = users_data.isnull().mean()

percent_missing
```

✓ 0.0s Open 'percent_missing' in Data Wrangler Python

user_id	0.000000
number_transactions	0.221173
total_amount_usd	0.221173
age	0.000000
job	0.006370
marital	0.000000
education	0.041076
default	0.000000
housing	0.000000
loan	0.000000
contact	0.287952
duration	0.000000
campaign	0.000000
pdays	0.000000
previous	0.000000
poutcome	0.817470
term_deposit	0.000000
date_joined	0.000664
device	0.002079

dtype: float64

- Check data null/nan if true will get the mean of data.
- If mean * 100, will be easier to read in percentage

```
#Generate a missing-value report
def missing_value_pct_df(data):
    """Create a data frame to summarize missing values."""

    percent_missing = data.isnull().mean()
    missing_value_df = \
pd.DataFrame(percent_missing).reset_index()

    missing_value_df = \
missing_value_df.rename(columns = {'index':'column_name',
...                               0: 'percent_missing'})

... # Multiply by 100 and round to 4 decimal places.
... missing_value_df['percent_missing'] = \
missing_value_df['percent_missing']. \
    apply(lambda x: round(x * 100, 2))

    missing_value_df = \
missing_value_df.sort_values(by = ['percent_missing'],
| | | | | | | | ascending = False)

    return missing_value_df
```

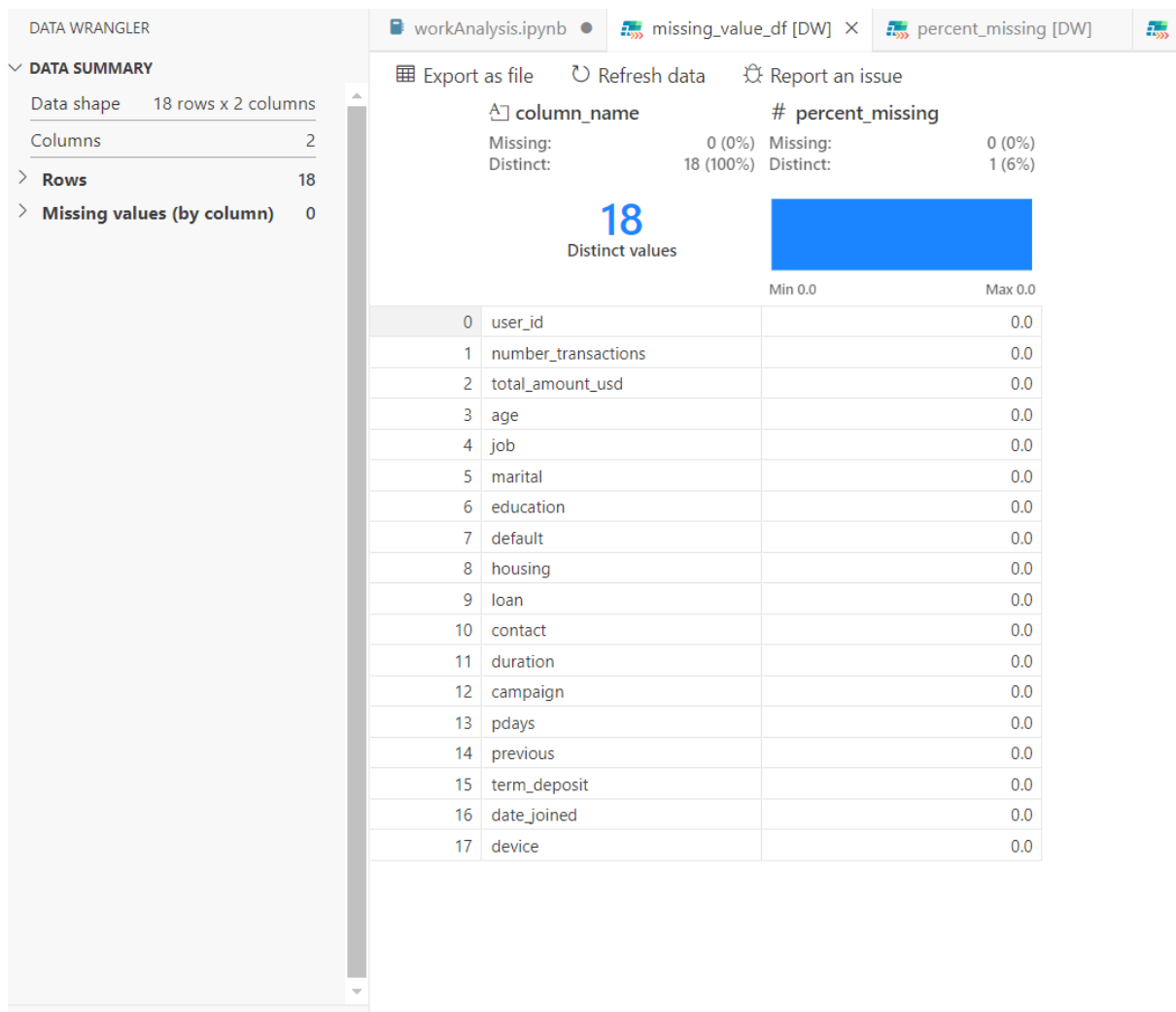
✓ 0.0s

```
missing_value_df = missing_value_pct_df(users_data)

missing_value_df
```

✓ 0.0s  Open 'missing_value_df' in Data Wrangler

- Create a function to get the missing value to all features



- Can view data from Data wrangler to easier to read in table with detail information

	column_name	percent_missing
15	poutcome	81.75
10	contact	28.80
1	number_transactions	22.12
2	total_amount_usd	22.12
6	education	4.11
4	job	0.64
18	device	0.21
17	date_joined	0.07
0	user_id	0.00
5	marital	0.00
3	age	0.00
9	loan	0.00
8	housing	0.00
7	default	0.00
11	duration	0.00
14	previous	0.00
13	pdays	0.00
12	campaign	0.00
16	term_deposit	0.00

- Based on data in table it showing some feature is zero. Means there no missing in data.
- The feature missing value with the number is showing that these features having the missing data
- The highest missing value is **poutcome** features

```
#Remove features with a high percentage of missing values

# Threshold above which to drop feature.

threshold = 80

cols_to_drop = \
list(missing_value_df[missing_value_df['percent_missing'] \
> threshold]['column_name'])

print('Number of features to drop:',
      missing_value_df[ \
missing_value_df['percent_missing'] > threshold].shape[0])

print(f'Features with missing values greater than {threshold}%:',
      cols_to_drop)
```

✓ 0.0s

Number of features to drop: 1
Features with missing values greater than 80%: ['poutcome']

- Put threshold = 80, which data more than threshold will be removed
- Feature **poutcome** is 81.75 is more than threshold.

```
users_data.info()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
Index: 45209 entries, 0 to 45215
Data columns (total 19 columns):
#   Column                Non-Null Count  Dtype
---  -
0   user_id                45209 non-null  object
1   number_transactions    35210 non-null  float64
2   total_amount_usd      35210 non-null  float64
3   age                   45209 non-null  int64
4   job                   44921 non-null  object
5   marital               45209 non-null  object
6   education              43352 non-null  object
7   default               45209 non-null  bool
8   housing               45209 non-null  bool
9   loan                  45209 non-null  bool
10  contact               32191 non-null  object
11  duration              45209 non-null  int64
12  campaign              45209 non-null  int64
13  pdays                 45209 non-null  int64
14  previous              45209 non-null  int64
15  poutcome              8252 non-null   object
16  term_deposit          45209 non-null  bool
17  date_joined           45179 non-null  datetime64[ns]
18  device                45115 non-null  object
dtypes: bool(4), datetime64[ns](1), float64(2), int64(5), object(7)
memory usage: 5.7+ MB
```

- Check original data info

```
users_data_cleaned = users_data.drop(cols_to_drop, axis = 1)
users_data_cleaned.info()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
Index: 45209 entries, 0 to 45215
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   user_id                45209 non-null  object
1   number_transactions    35210 non-null  float64
2   total_amount_usd      35210 non-null  float64
3   age                    45209 non-null  int64
4   job                    44921 non-null  object
5   marital                45209 non-null  object
6   education              43352 non-null  object
7   default                45209 non-null  bool
8   housing                45209 non-null  bool
9   loan                   45209 non-null  bool
10  contact                32191 non-null  object
11  duration                45209 non-null  int64
12  campaign               45209 non-null  int64
13  pdays                 45209 non-null  int64
14  previous               45209 non-null  int64
15  term_deposit           45209 non-null  bool
16  date_joined            45179 non-null  datetime64[ns]
17  device                 45115 non-null  object
dtypes: bool(4), datetime64[ns](1), float64(2), int64(5), object(6)
memory usage: 5.3+ MB
```

- Check new data after cleaned info
- The **poutcome** in new data removed

```
# Confirm feature was dropped.
```

```
missing_value_df = missing_value_pct_df(users_data_cleaned)
```

```
missing_columns = \
list(missing_value_df[missing_value_df['percent_missing'] \
> 0]['column_name'])
print(missing_columns)
print('Number of features with missing values:',
      len(missing_columns))
```

✓ 0.0s

```
['contact', 'number_transactions', 'total_amount_usd', 'education', 'job', 'device', 'date_joined']
Number of features with missing values: 7
```

- Show list column name with missing value if more than 0
- Show the total number column with the missing value

```
#Identify numerical data with missing values
dtypes = ['int64', 'float64']

numerical_columns = \
list(users_data_cleaned.select_dtypes(dtypes).columns)

print('Numerical features with missing values:',
      list(set(numerical_columns).intersection(missing_columns)))
```

✓ 0.0s

Numerical features with missing values: ['number_transactions', 'total_amount_usd']

- Show list column name with missing for numerical value only

```
#Impute missing data values for total_amount_usd
# Find a sample user with missing value.

sample_user = \
users_data_cleaned[users_data_cleaned['total_amount_usd']. \
isnull()].sample(1).user_id

sample_user
```

✓ 0.0s  Open 'sample_user' in Data Wrangler

44046 284279c5-e984-4274-be31-8c04b2033f93

Name: user_id, dtype: object

- Select 1 random user id with missing value for 'total_amount_usd'.
- Example , **total_amount_usd** = NaN, this data will selected and show user id

```
print(users_data_cleaned['total_amount_usd'].isnull().value_counts())
print(f'total null in column total_amount_usd-{users_data_cleaned['total_amount_usd'].isnull().sum()}')
```

✓ 0.0s

```
total_amount_usd
False    35210
True      9999
Name: count, dtype: int64
total null in column total_amount_usd-9999
```

- List data and total count based on value null for **total_amount_usd**, if null then true else will be false
- Show total of **total_amount_usd** if null

```
users_data_cleaned[users_data_cleaned.user_id=='602a3ab8-4d6b-4877-b74e-74e39d54560f']
```

✓ 0.0s

Python

	user_id	number_transactions	total_amount_usd	age	job	marital	education	default	housing	loan	contact	duration	campaign	pdays	
32236	602a3ab8-4d6b-4877-b74e-74e39d54560f		NaN	NaN	31	technician	single	tertiary	False	True	False	cellular	60	1	342

- Pick data based on user id
- **total_amount_usd** show NaN

```

# Print mean of total_amount_usd.

print('Mean total_amount_usd:',
      round(users_data_cleaned['total_amount_usd'].mean(), 2))

# Impute missing values for total_amount_usd with mean.

users_data_cleaned['total_amount_usd']. \
fillna(round(users_data_cleaned['total_amount_usd'].mean(), 2),
       inplace = True)

```

✓ 0.0s

Mean total_amount_usd: 1369.42

- find mean from **total_amount_usd** and round by 2 decimals
- Impute the missing value by mean of **total_amount_usd**

```

users_data_cleaned[users_data_cleaned.user_id=='602a3ab8-4d6b-4877-b74e-74e39d54560f']

```

✓ 0.0s

	user_id	number_transactions	total_amount_usd	age	job	marital	education	default	housing	loan	contact	duration	campaign	pdays
32236	602a3ab8-4d6b-4877-b74e-74e39d54560f	NaN	1369.42	31	technician	single	tertiary	False	True	False	cellular	60	1	342

- Pick data based on user id
- **total_amount_usd** not show **NaN** anymore and replacing with **mean** value

```

print(users_data_cleaned['total_amount_usd'].isnull().value_counts())
print(f'total null in column total_amount_usd-{users_data_cleaned["total_amount_usd"].isnull().sum()}')

```

✓ 0.0s

```

total_amount_usd
False    45209
Name: count, dtype: int64
total null in column total_amount_usd-0

```

- verify the value of null from **total_amount_usd** is null still there or not

```

users_data_cleaned[users_data_cleaned. \
                    user_id.isin(sample_user)][ 'total_amount_usd' ]

```

✓ 0.0s

```

44046    1369.42
Name: total_amount_usd, dtype: float64

```

- verify the value has been replace for **total_amount_usd** from variable **sample_user**
- **sample_user** hold value 1 random data **user_id** with null/nan value for **total_amount_usd**
- find data that match in **sample_user**

```
users_data_cleaned.number_transactions.value_counts()
```

✓ 0.0s

number_transactions

```
2.0    14853
3.0     9909
4.0     4010
5.0     1223
1.0       662
6.0       500
7.0       337
15.0      324
16.0      315
20.0      300
19.0      294
13.0      294
11.0      294
17.0      277
14.0      272
10.0      272
12.0      271
18.0      270
9.0       268
8.0       265
```

Name: count, dtype: int64

- List data in **number_transactions** and count for each value

```
#Replace missing values for number_transactions with 0
users_data_cleaned['number_transactions']. \
    fillna(0, inplace = True)
```

✓ 0.0s

```
users_data_cleaned[users_data_cleaned. \
    user_id.isin(sample_user)]['number_transactions']
```

✓ 0.0s

44046 0.0

Name: number_transactions, dtype: float64

- Impute the missing value for **number_transactions** with 0 value
- Find data with matching data from variable **sample_user**

✓ 0.0s

- List data in **number_transactions** and count for each value
- Verify data with missing value that supposed to replace with 0 value.

Categorical features with missing values: ['contact', 'job', 'education', 'device']

- Generate Code Markdown

55] ✓ 0.0s

- List data in **contact** and count for each value

```
users_data_cleaned.device.value_counts()
```

✓ 0.0s

```
device
mobile      27504
desktop     12112
tablet       5499
Name: count, dtype: int64
```

- List data in **device** and count for each value

```
users_data_cleaned.education.value_counts()
```

✓ 0.0s

```
education
secondary  23202
tertiary   13300
primary    6850
Name: count, dtype: int64
```

Generate + Cells + Markdown

- List data in **education** and count for each value

```
users_data_cleaned.job.value_counts()
```

✓ 0.0s

```
job
blue-collar    9731
management     9457
technician     7597
admin.         5171
services       4154
retired        2264
self-employed  1579
entrepreneur   1487
unemployed     1303
housemaid      1240
student         938
Name: count, dtype: int64
```

- List data in **job** and count for each value

```
#Replace categorical missing values with 'Unknown'
users_data_cleaned.device.fillna('Unknown', inplace = True)
users_data_cleaned.education.fillna('Unknown', inplace = True)
users_data_cleaned.contact.fillna('Unknown', inplace = True)
users_data_cleaned.job.fillna('Unknown', inplace = True)
```

[59] ✓ 0.0s

```
users_data_cleaned.job.value_counts()
```

[60] ✓ 0.0s

```
... job
blue-collar    9731
management     9457
technician     7597
admin.         5171
services       4154
retired        2264
self-employed  1579
entrepreneur   1487
unemployed     1303
housemaid      1240
student         938
Unknown        288
Name: count, dtype: int64
```

- Impute data missing value with 'Unknown' value for **device, education, contact, job**
- Verify value that has been replace with 'Unknown' value

```

#Check if there are any other missing values
missing_value_df = missing_value_pct_df(users_data_cleaned)
missing_columns = \
list(missing_value_df[missing_value_df['percent_missing'] \
| > 0]['column_name'])

print('Number of features with missing values:', len(missing_columns))
print('Features with missing values:', missing_columns)

```

[61] ✓ 0.0s

```

... Number of features with missing values: 1
Features with missing values: ['date_joined']

```

```

#Remove all rows where date_joined is missing
print('Number of users with corrupted data:',
      users_data_cleaned[users_data_cleaned['date_joined']. \
| isnull()].shape[0])

```

[62] ✓ 0.0s

```

... Number of users with corrupted data: 30

```

- Check features with missing value, found left 1 only. The feature is **date_joined**
- Check shape of data but row only. The data have 30 missing values for **date_joined**

```

# Remove corrupted data.

users_data_cleaned = \
users_data_cleaned[~users_data_cleaned['date_joined'].isnull()]

```

✓ 0.0s

```

# Check to see if any corrupted rows remain.

print('Number of users with corrupted data:',
      users_data_cleaned[users_data_cleaned['date_joined']. \
| isnull()].shape[0])

```

✓ 0.0s

```

Number of users with corrupted data: 0

```

- Remove the missing data for feature **date_joined**
- Check shape of data but row only. The no data missing values for **date_joined**

```

#Perform one last check for missing values
missing_value_df = missing_value_pct_df(users_data_cleaned)
missing_columns = \
list(missing_value_df[missing_value_df['percent_missing'] \
| > 0]['column_name'])

print('Number of features with missing values:',
      len(missing_columns))

```

✓ 0.0s

```

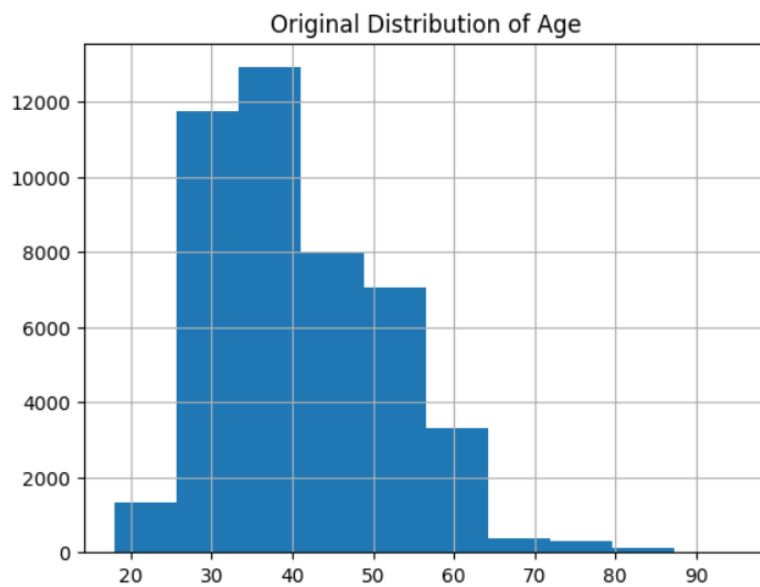
Number of features with missing values: 0

```

- Check features and no more the feature with missing values

```
#View the distribution of age
users_data_cleaned['age'].hist()
plt.title('Original Distribution of Age');
```

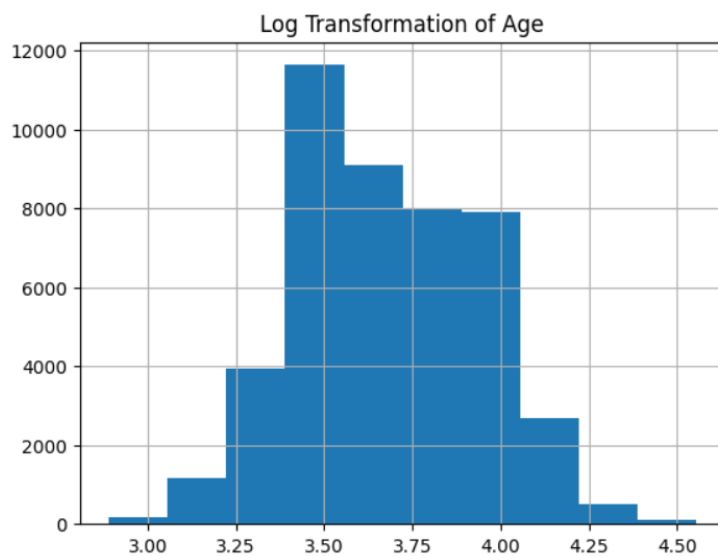
✓ 0.1s



- Use matplotlib library to create graph histogram for **age**
- The graph shows positive skewness

```
#Apply a log transformation to age
np.log(users_data_cleaned['age']).hist()
plt.title('Log Transformation of Age');
```

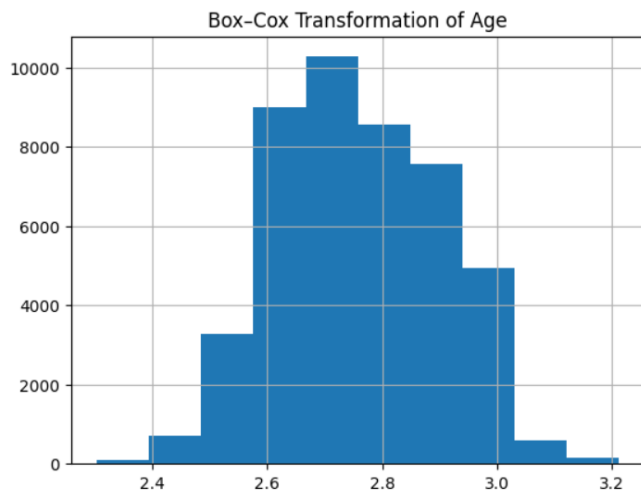
✓ 0.1s



- Use matplotlib library to create graph histogram for **age** with applying log transformation
- The graph shows almost normal skew, but log transformation not that very effectively
- But can use square root and box – cox transformation

```
#Apply a Box-Cox transformation to age
pd.Series(stats.boxcox(users_data_cleaned['age'])[0]).hist()
plt.title('Box-Cox Transformation of Age');
```

68] ✓ 0.7s



- Use matplotlib library to create graph histogram for **age** with Box-Cox transformation
- The graph shows normal skew

```
#Identify categorical features
categorical_columns = \
list(users_data_cleaned.select_dtypes(['object']).columns)

print('The number of categorical features:',
      len(categorical_columns))
print('The names of categorical features:',
      categorical_columns)
```

✓ 0.0s

The number of categorical features: 6
The names of categorical features: ['user_id', 'job', 'marital', 'education', 'contact', 'device']

- Get the total feature from object datatype
- List the name of features

```
#One-hot encode job
users_data_cleaned.job.value_counts()
```

✓ 0.0s

```
job
blue-collar    9725
management    9453
technician     7592
admin.         5168
services       4152
retired        2262
self-employed  1577
entrepreneur   1485
unemployed     1301
housemaid      1239
student        937
Unknown        288
Name: count, dtype: int64
```

- Get the total value category in feature **job**

```

# Create object for one-hot encoding.

encoder = ce.OneHotEncoder(cols = 'job',
                             return_df = True,
                             use_cat_names = True)

```

✓ 0.0s Python

- Use one hot encoder to create new features from value category in feature **job**
- The data will be in data frame
- The value would be true and false but in numerical value which is 1 and 0

```

# Fit and transform data.

users_data_encoded = encoder.fit_transform(users_data_cleaned)

# Preview the data.

users_data_encoded.head()

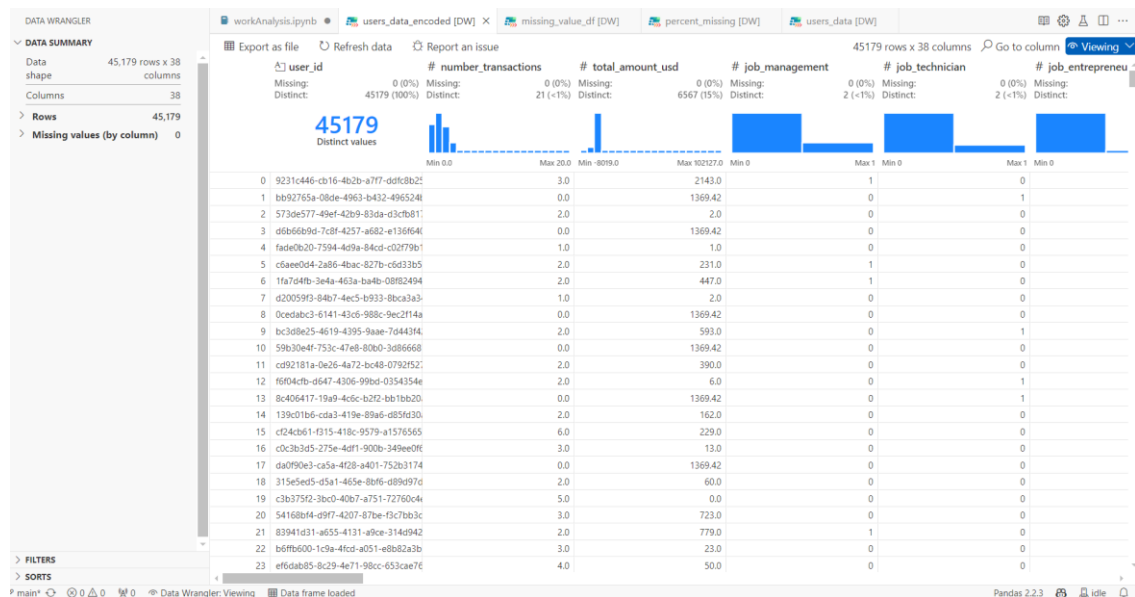
```

✓ 0.2s Open 'users_data_encoded' in Data Wrangler Python

	user_id	number_transactions	total_amount_usd	age	job_management	job_technician	job_entrepreneur	job_blue-collar	job_Unknown	job_retired	...	hou
0	9231c446-cb16-4b2b-a7f7-ddfc8b25aaf6	3.0	2143.00	58	1	0	0	0	0	0	...	
1	bb92765a-08de-4963-b432-496524b39157	0.0	1369.42	44	0	1	0	0	0	0	...	
2	573de577-49ef-42b9-83da-d3cfb817b5c1	2.0	2.00	33	0	0	1	0	0	0	...	
3	d6b66b9d-7c8f-4257-a682-e136f640b7e3	0.0	1369.42	47	0	0	0	1	0	0	...	
4	fade0b20-7594-4d9a-84cd-c02f79b1b526	1.0	1.00	33	0	0	0	0	1	0	...	

5 rows × 29 columns

- Encoder – one hot encoder
- Function **fit_transform**, fit will study the data for feature and category data need to encode and do transform data into binary
- The data will be store in variable **users_data_encoded**



- Open data in data wrangler to easier to read in table with details information

```

list(users_data_encoded)

```

Python

```

['user_id',
 'number_transactions',
 'total_amount_usd',
 'age',
 'job_management',
 'job_technician',
 'job_entrepreneur',
 'job_blue-collar',
 'job_Unknown',
 'job_retired',
 'job_admin.',
 'job_services',
 'job_self-employed',
 'job_unemployed',
 'job_housemaid',
 'job_student',
 'marital',
 'education',
 'default',
 'housing',
 'loan',
 'contact',
 'duration',
 'campaign',
 'pdays',
 'previous',
 'term_deposit',
 'date_joined',
 'device']

```

- List features in `users_data_encoded`

```

print('Shape of data before encoding:',
      users_data_cleaned.shape)
print('Shape of data after encoding:',
      users_data_encoded.shape)

```

Python

```

Shape of data before encoding: (45179, 18)
Shape of data after encoding: (45179, 29)

```

- Check and compare shape of data before and after encoding

```

#Dummy encode marital
marital_encoded = \
pd.get_dummies(data = users_data_encoded['marital'],
               drop_first = True,
               dtype = int)

marital_encoded

```

Python

	married	single
0	1	0
1	0	1
2	1	0
3	1	0
4	0	1
...	...	...
45211	1	0
45212	0	0
45213	1	0
45214	1	0
45215	1	0

45179 rows × 2 columns

- Same method to one hot encoding, but this time use pandas library

```
# Concatenate the new encoded columns.

users_data_encoded = \
pd.concat([users_data_encoded, marital_encoded], axis = 1)

# Drop the original variable.

users_data_encoded.drop(['marital'], axis = 1, inplace = True)

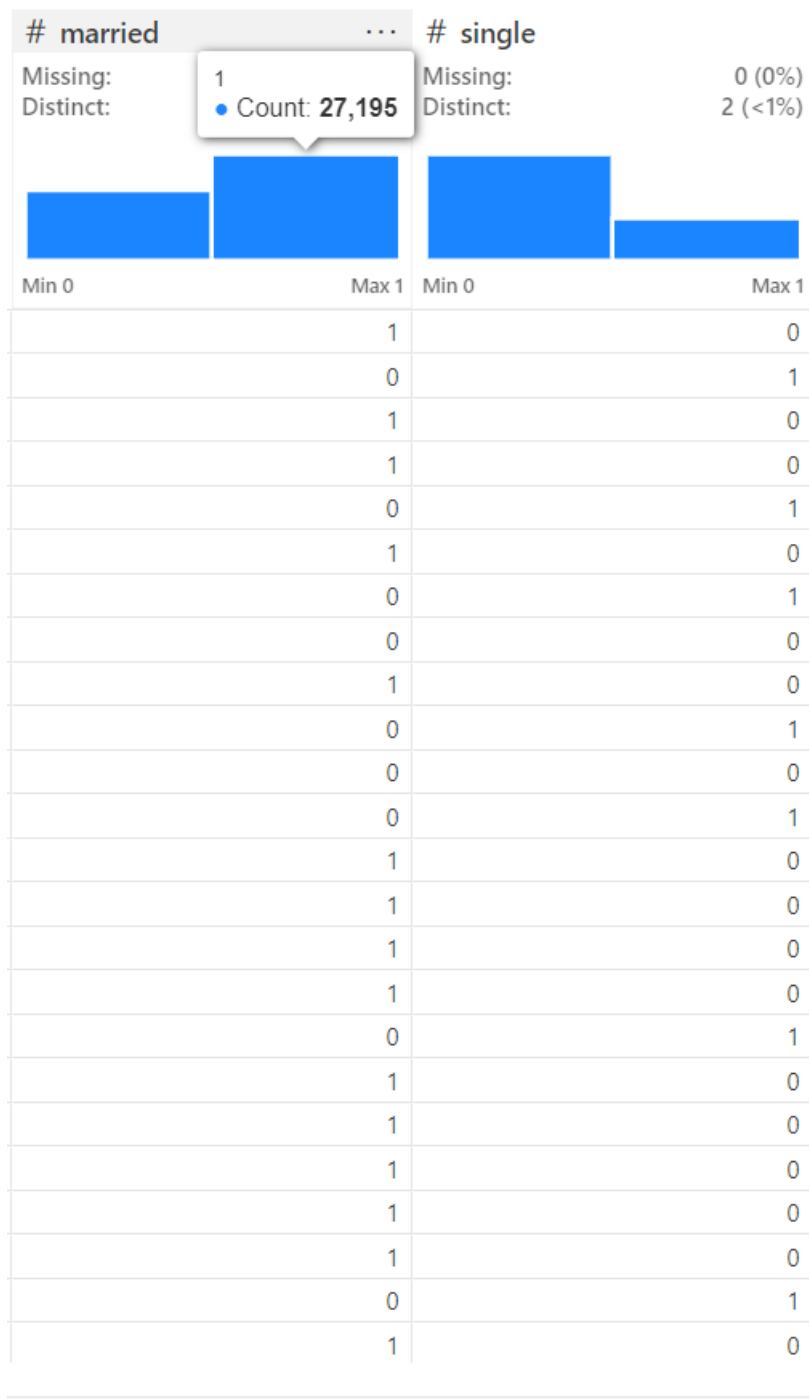
# Preview the data.

users_data_encoded.head()
```

✓ 0.0s Open 'users_data_encoded' in Data Wrangler Python

	user_id	number_transactions	total_amount_usd	age	job_management	job_technician	job_entrepreneur	job_blue-collar	job_Unknown	job_retired	...	co
0	9231c446-cb16-4b2b-a7f7-ddfc8b25aaf6	3.0	2143.00	58	1	0	0	0	0	0	...	Unk
1	bb92765a-08de-4963-b432-496524b39157	0.0	1369.42	44	0	1	0	0	0	0	...	Unk
2	573de577-49ef-42b9-83da-d3cfb817b5c1	2.0	2.00	33	0	0	1	0	0	0	...	Unk
3	d6b66b9d-7c8f-4257-a682-e136f640b7e3	0.0	1369.42	47	0	0	0	1	0	0	...	Unk

- Concatenate the new encoded feature for **marital**
- Remove the original feature **marital**



- Open data wrangler to easier read in table for new feature **marital**

```
print('Shape of data after encoding:',  
      users_data_encoded.shape)
```

```
list(users_data_encoded)
```

Shape of data after encoding: (45179, 30)

```
['user_id',  
 'number_transactions',  
 'total_amount_usd',  
 'age',  
 'job_management',  
 'job_technician',  
 'job_entrepreneur',  
 'job_blue-collar',  
 'job_Unknown',  
 'job_retired',  
 'job_admin.',  
 'job_services',  
 'job_self-employed',  
 'job_unemployed',  
 'job_housemaid',  
 'job_student',  
 'education',  
 'default',  
 'housing',  
 'loan',  
 'contact',  
 'duration',  
 'campaign',  
 'pdays',  
 'previous',  
 'term_deposit',  
 'date_joined',  
 'device',  
 'married',  
 'single']
```

- Check the shape of data to get the rows number and column numbers
- List the feature with new feature encoding for **job** and **marital**

```
#One-hot encode the remaining categorical variables
cols = ['education', 'contact', 'device']

encoder = ce.OneHotEncoder(cols = cols,
                            return_df = True,
                            use_cat_names = True)
```

78] ✓ 0.0s Python

```
# Fit and transform data.

users_data_encoded = encoder.fit_transform(users_data_encoded)

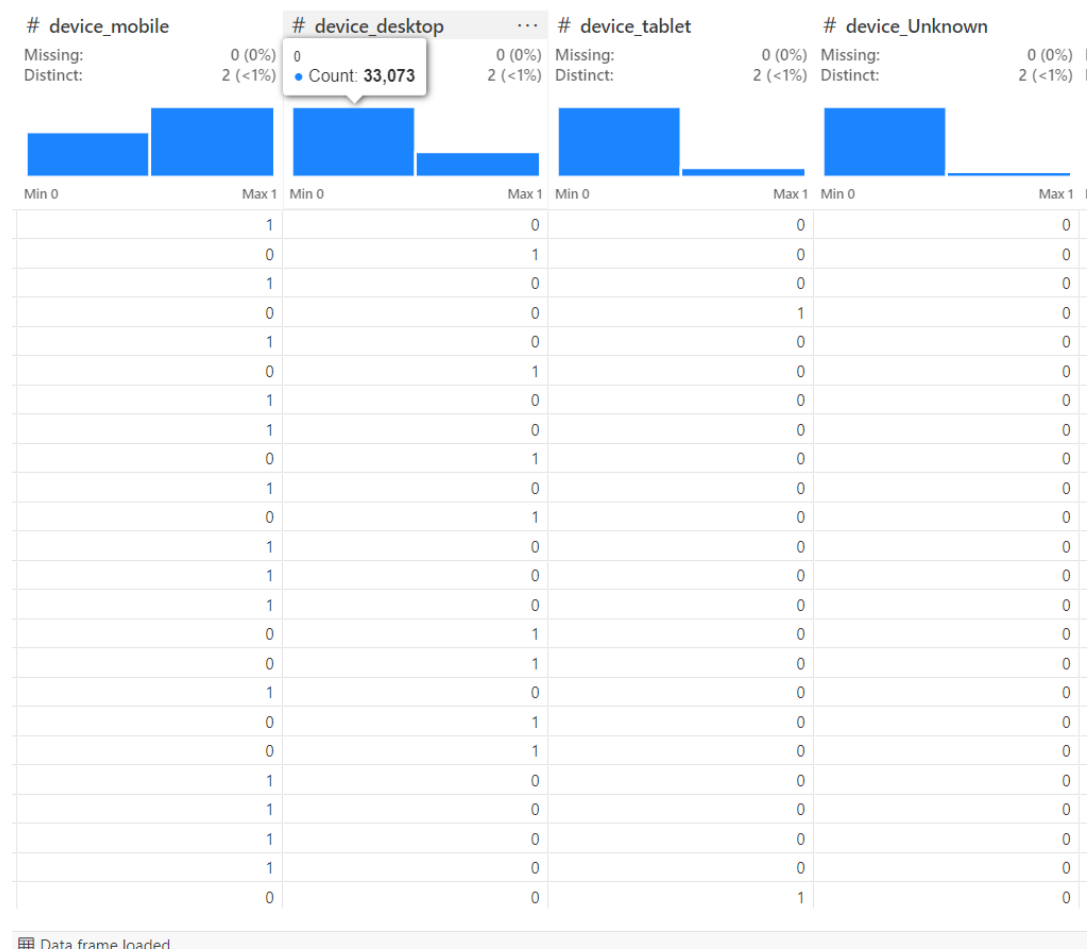
# Preview the data.

users_data_encoded.head()
```

79] ✓ 0.4s Open 'users_data_encoded' in Data Wrangler Python

	user_id	number_transactions	total_amount_usd	age	job_management	job_technician	job_entrepreneur	job_blue-collar	job_Unknown	job_retired	...	pda
0	9231c446-cb16-4b2b-a7f7-ddfc8b25aaf6	3.0	2143.00	58	1	0	0	0	0	0	...	
1	bb92765a-08de-4963-b432-496524b39157	0.0	1369.42	44	0	1	0	0	0	0	...	
2	573de577-49ef-42b9-83da-d3cfb817b5c1	2.0	2.00	33	0	0	1	0	0	0	...	

- This time will do for multiple features and do encoding together
- Do fit and transform for multiple features – **education, contact, device**



- Open data wrangler to easier read in table for new feature **device**

```
# contact_telephone
```

0 (0%)

2 (<1%)

[illegible]

- Open data wrangler to easier read in table for new feature **contact**


```
# Define age bins and labels.

bins = [18, 25, 35, 45, 55, 65, 75, 110]
labels = ['18-24', '25-34', '35-44',
          '45-54', '55-64', '65-74', '75+']

# Perform binning using bin list.

users_data_encoded['age_group'] = \
pd.cut(users_data_encoded['age'], bins = bins,
        labels = labels, right = False)

# Map bins to integer values.
users_data_encoded['age_group_encoded'] = \
users_data_encoded['age_group'].cat.codes
```

✓ 0.0s

```
# Verify correct binning.

age_vars = ['age_group_encoded', 'age_group', 'age']

users_data_encoded[age_vars].sample(10)
```

✓ 0.0s

- Based on standard for min 18,max = 95 and standard deviation = 10
- We declare the bins [18, 25, 35, 45, 55, 65, 75, 110]
- Add new features **age_group** for bins
- Data **age**, data will be divided following the bins, and add label for that category.
- Right = false -> in the range before the right value. Example 18-25. Means included 18 until 24
- Add new features for code category after do bin for **age**. This feature will be represented as int by using **cat.codes**
- Verify the value **age_group_encoded**, **age_group** and **age**

	age_group_encoded	age_group	age
33564	1	25-34	33
36975	1	25-34	28
42792	3	45-54	52
19170	4	55-64	56
37634	1	25-34	31
3951	1	25-34	32
34320	4	55-64	58
4718	2	35-44	36
41321	1	25-34	28
44008	1	25-34	31

- The sample 10 random of data

```
#Plot the new distribution of age
user_age_dist = users_data_encoded.age_group.value_counts()
```

```
user_age_dist
```

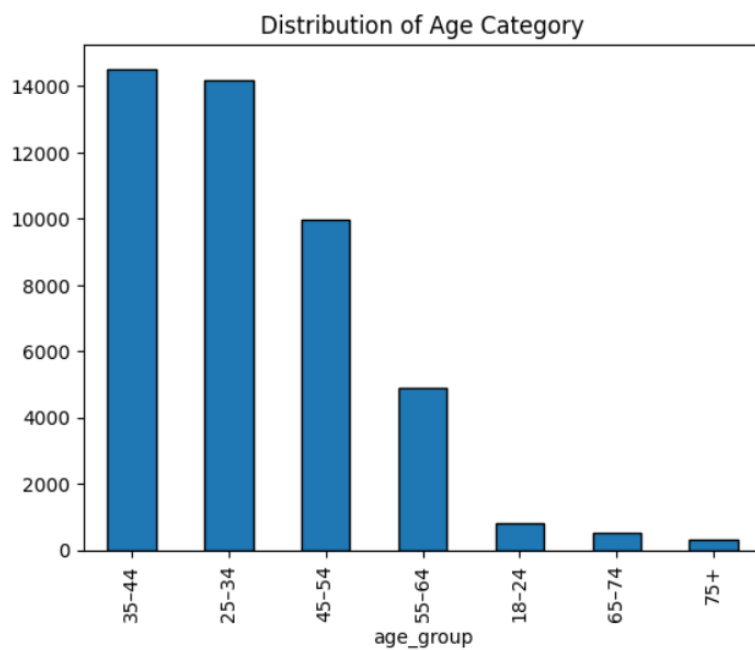
✓ 0.0s Open 'user_age_dist' in Data Wrangler

```
age_group
35-44    14524
25-34    14194
45-54     9951
55-64     4892
18-24      809
65-74      510
75+        299
Name: count, dtype: int64
```

- Check the value category of **age** and count for each category
- **age_group_dist** will hold this value

```
user_age_dist.plot(kind = 'bar', edgecolor = 'black')
plt.title('Distribution of Age Category');
```

✓ 0.1s



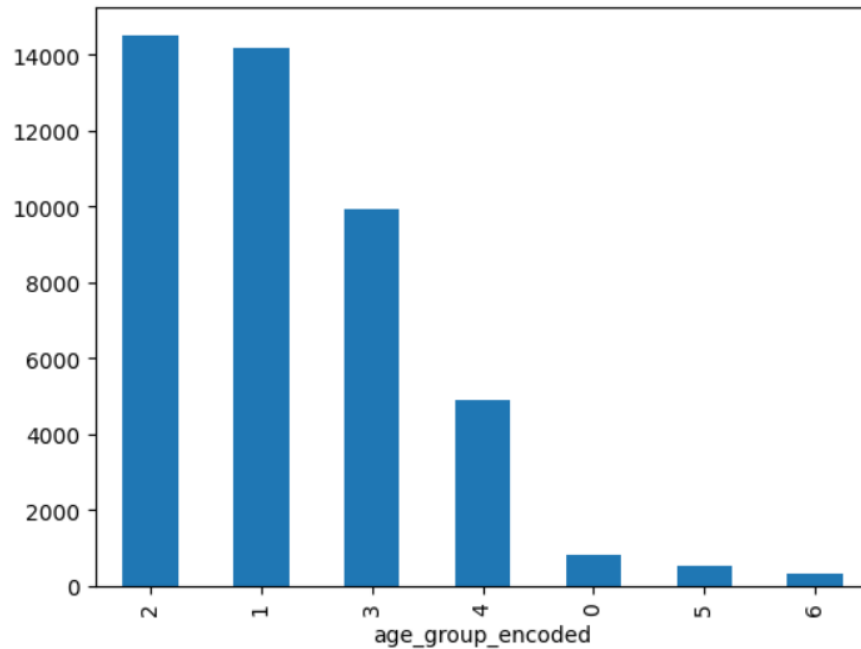
- Create a graph bar for variable **age_group_dist** with the total of each category

36]

```
# Check against encoded values.

users_data_encoded.age_group_encoded. \
value_counts().plot(kind = 'bar');
```

✓ 0.1s



- Create a graph bar for variable **age_group_encoded** with the total of each category but the category will display as code.

```
#Drop the age and age_group variables
users_data_encoded.drop(['age', 'age_group'],
| | | | | axis = 1, inplace = True)

list(users_data_encoded)
```

✓ 0.0s

- the features **age** and **age_group** will not use anymore since **age_group_encoded** and represented as int is more useable when it coming to Machine learning process.

```
[
    'user_id',
    'number_transactions',
    'total_amount_usd',
    'job_management',
    'job_technician',
    'job_entrepreneur',
    'job_blue-collar',
    'job_Unknown',
    'job_retired',
    'job_admin.',
    'job_services',
    'job_self-employed',
    'job_unemployed',
    'job_housemaid',
    'job_student',
    'education_tertiary',
    'education_secondary',
    'education_Unknown',
    'education_primary',
    'default',
    'housing',
    'loan',
    'contact_Unknown',
    'contact_cellular',
    'contact_telephone',
    ...
    'device_tablet',
    'device_Unknown',
    'married',
    'single',
    'age_group_encoded']
```

- show the new list features

```
#Create a month_joined variable from date_joined
users_data_encoded['month_joined'] = \
    users_data_encoded.date_joined.dt.month
```

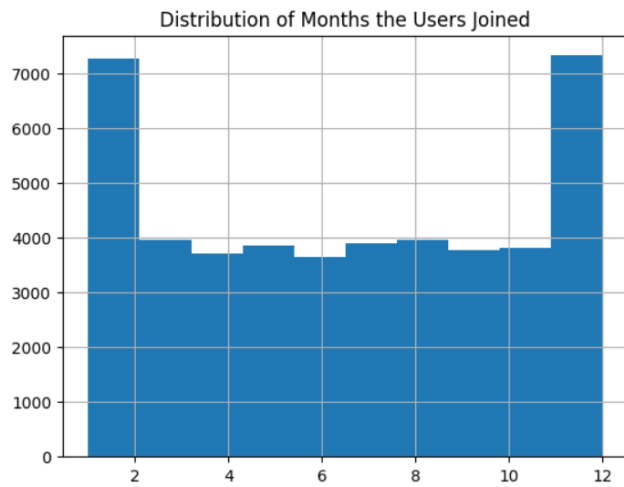
✓ 0.0s

- add new data for month only from **date_joined** using **dt.month**

```
# View the distribution of data.

users_data_encoded['month_joined'].hist()
plt.title('Distribution of Months the Users Joined');
```

✓ 0.1s



- create a graph histogram for months of **date_joined**

```
#Drop date_joined
users_data_encoded.drop(['date_joined'],
                        axis = 1, inplace = True)

list(users_data_encoded)
```

✓ 0.0s

- removed **date_joined**

```
['user_id',
 'number_transactions',
 'total_amount_usd',
 'job_management',
 'job_technician',
 'job_entrepreneur',
 'job_blue-collar',
 'job_Unknown',
 'job_retired',
 'job_admin.',
 'job_services',
 'job_self-employed',
 'job_unemployed',
 'job_housemaid',
 'job_student',
 'education_tertiary',
 'education_secondary',
 'education_Unknown',
 'education_primary',
 'default',
 'housing',
 'loan',
 'contact_Unknown',
 'contact_cellular',
 'contact_telephone',
 ...,
 'device_Unknown',
 'married',
 'single',
 'age_group_encoded',
 'month_joined']
```

- show list features
- only keep new feature **month_joined**

```
#Remove features with low variance
users_data_encoded.std(numeric_only = True)
```

✓ 0.0s

```
number_transactions      3.749994
total_amount_usd        2704.291321
job_management           0.406767
job_technician           0.373908
job_entrepreneur         0.178296
job_blue-collar         0.411004
job_Unknown              0.079587
job_retired              0.218087
job_admin.               0.318287
job_services             0.288889
job_self-employed       0.183543
job_unemployed           0.167236
job_housemaid            0.163318
job_student              0.142513
education_tertiary       0.455691
education_secondary      0.499832
education_Unknown        0.198480
education_primary        0.358591
default                  0.133095
housing                  0.496878
loan                     0.366802
contact_Unknown          0.452851
contact_cellular         0.477695
contact_telephone        0.245250
duration                 257.281109
...
married                  0.489504
single                   0.450412
age_group_encoded        1.106811
month_joined             3.428374
dtype: float64
```

- show standard deviation to all features but numeric only

```
# Define standard deviation threshold.

threshold = 0.1

# Identify features below threshold.

cols_to_drop = \
list(users_data_encoded. \
     std(numeric_only = True)[users_data_encoded. \
                              std(numeric_only = True) \
                              < threshold].index.values)

print('Features with low standard deviation:',
      cols_to_drop)
```

✓ 0.0s

Features with low standard deviation: ['job_Unknown', 'device_Unknown']

- show standard deviation to all features but numeric only but if below than threshold
- threshold = 0.1
- 2 features listed. **Job_Unknown, device_Unknown**

✓ 0.0s

- Check the higher correlation between features for numerical only and above the threshold that has been set to 0.75

- Why we need to check the highest **corr** value and put threshold , because we need to avoid the data that have a duplicate value and will make the model simple and overfitting
- List features correlation with together the **corr** value.

Tidy up the output.

```
record_collinear = pd.DataFrame(high_corr_var). \
    rename(columns = {0: 'drop_feature',
                      1: 'corr_feature',
                      2: 'corr_values'})

record_collinear = record_collinear. \
    sort_values(by = 'corr_values', ascending = False)

record_collinear = record_collinear.reset_index(drop = True)

record_collinear
```

Python

	drop_feature	corr_feature	corr_values
0	contact_Unknown	contact_cellular	0.86
1	married	single	0.77
2	device_mobile	device_desktop	0.75

- show features correlation with together the **corr** value in dataframe.
- List reset index and sort **corr** values in ascending

	drop_feature	corr_feature	corr_values
0	contact_Unknown	contact_cellular	0.86
1	married	single	0.77
2	device_mobile	device_desktop	0.75

Generate

+ Code

+ Markdown

```
cols_to_drop = list(record_collinear['drop_feature'])
print(cols_to_drop)
```

```
['contact_Unknown', 'married', 'device_mobile']
```

```
users_data_final = users_data_interim.drop(cols_to_drop,
                                           axis = 1)

list(users_data_final)
```

Python

- Drop the features with highest **corr** value
- The features were **contact_Unknown, married, device_mobile**

```
[
    'user_id',
    'number_transactions',
    'total_amount_usd',
    'job_management',
    'job_technician',
    'job_entrepreneur',
    'job_blue-collar',
    'job_retired',
    'job_admin.',
    'job_services',
    'job_self-employed',
    'job_unemployed',
    'job_housemaid',
    'job_student',
    'education_tertiary',
    'education_secondary',
    'education_Unknown',
    'education_primary',
    'default',
    'housing',
    'loan',
    'contact_cellular',
    'contact_telephone',
    'duration',
    'campaign',
    ...
    'device_desktop',
    'device_tablet',
    'single',
    'age_group_encoded',
    'month_joined']
```

- List the feature.
- Check and verify the feature that have been drop

```
#Filter by demographics data
users_data_demographics = \
users_data_final.filter(regex = 'education|job|age|single')

users_data_demographics.head(n = 3)
```

Python

	job_management	job_technician	job_entrepreneur	job_blue-collar	job_retired	job_admin.	job_services	job_self-employed	job_unemployed	job_housemaid	job_student
0	1	0	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0	0	0
2	0	0	1	0	0	0	0	0	0	0	0

- show the features with filter the feature that have name like 'education,job,age,single' in table data frame

```
#Standardize the demographics data
users_data_demographics.describe()
```

Python

	job_management	job_technician	job_entrepreneur	job_blue-collar	job_retired	job_admin.	job_services	job_self-employed	job_unemployed	job_housemaid
count	45179.000000	45179.000000	45179.000000	45179.000000	45179.000000	45179.000000	45179.000000	45179.000000	45179.000000	45179.000000
mean	0.209234	0.168043	0.032869	0.215255	0.050068	0.114389	0.091901	0.034906	0.028797	0.02742
std	0.406767	0.373908	0.178296	0.411004	0.218087	0.318287	0.288889	0.183543	0.167236	0.16331
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
50%	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
75%	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
max	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000

- Check info data about count,unique,top,frequent,mean,min,25%,50%,75%, max and standard for all features

```
scaler = StandardScaler()

scaler.fit(users_data_demographics)
users_data_scaled = scaler.transform(users_data_demographics)

print('New standard deviation: ', users_data_scaled.std())
print('New mean: ', round(users_data_scaled.mean()))
```

New standard deviation: 1.0
New mean: 0

- Use **sklearn.preprocessing** to use standardscaler function
- **fit()** – calculate mean and standard for scaling
- **transform()** - Standardizes the data
- check new mean and standard

```
#Perform PCA to reduce the dimensionality of the demographics dataset
pca = PCA(n_components = 2, random_state = 1)

pca.fit(users_data_scaled)

reduced = pca.transform(users_data_scaled)

reduced_df = pd.DataFrame(reduced, columns = ['PCA1', 'PCA2'])

reduced_df
```

	PCA1	PCA2
0	2.555464	1.080715
1	-0.818581	-1.751929
2	-0.572830	-0.553687
3	-0.546107	1.596294
4	0.385254	-0.614489
...	...	...
45174	1.159487	0.074663
45175	-0.656264	4.368307
45176	-1.324414	2.365734
45177	-1.577706	1.041209
45178	-0.628998	-0.137230

45179 rows × 2 columns

- use PCA to reduce the dimensionality
- to avoid duplicate data
- Reducing Overfitting

- help improve model performance; this leads to better generalization on unseen data. Will make it perform well in new data
- unseen data—data that wasn't part of the training process. This capability is crucial because the ultimate goal of a model is to make accurate predictions on real-world data, not just the data it was trained on.
- Enhancing Interpretability; make it easier to understand the relationship within data
- sklearn.decomposition library for PCA
- PCA(n_components=2, random_state=1), will return 2 column in dataframe

```

4] reduced_df.info()
Python

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45179 entries, 0 to 45178
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype  
---  -
 0   PCA1    45179 non-null   float64
 1   PCA2    45179 non-null   float64
dtypes: float64(2)
memory usage: 706.1 KB

5] users_data_final.to_pickle('users_data_final.pickle')
Python

#Load the demographics dataset with PCA applied
reduced_df.to_pickle('users_data_demo_pca.pickle')
Python
6]

```

- Check data info from PCA dataframe
- Save the data into pickle file